

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Beyond Carving: Deterministic Deleted File Recovery in Btrfs

KRISH PANDEY¹, MISHA JAIN¹, NISHA P. SHETTY¹

¹ Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka, India-576104

Corresponding author: Nisha P Shetty (e-mail: nisha.pshetty@manipal.edu).

The publication charge of this article is covered by Manipal Academy of Higher Education, Manipal 576104, Karnataka, India

ABSTRACT This paper presents a novel offline deleted-file recovery algorithm for the B-tree File System (Btrfs) that operates by fully parsing the raw filesystem image. Btrfs is a copy-on-write (CoW) filesystem whose on-disk intricacy greatly complicates post-deletion data recovery. Existing recovery approaches typically rely on heuristic carving, incomplete metadata interpretation, or live filesystem artifacts, yielding low integrity guarantees and limited origin reconstruction. The algorithm reconstructs filesystem state by traversing Btrfs trees directly (as the driver does, with fine-tuned changes to aid recovery), correlating extents, inodes, and historical metadata to recover deleted regular files with verifiable integrity, deterministic deletion classification, and reconstructed file origin, compared to existing methods. Carving-based recovery has evolved significantly, yet its fundamental reliance on file signatures means that it cannot reconstruct the provenance of the filesystem, file paths, or inode metadata. Unlike signature-based carving, the extraction of data via orphan items, or similar methods, the proposed approach attempts to reconstruct both the current and the historical file-system layout using native Btrfs structures, significantly improving the fidelity of file recovery and the structural transparency of failure cases and enabling intact recovery of file content, where extents remain resolvable through the chunk tree, alongside full recovery of associated metadata and paths. In addition, the algorithm extracts file-creation and access metadata and file paths sufficient for forensic timeline reconstruction. The paper further analyzes key Btrfs design features, such as Btrfs leaves, nodes, extents, and their attributes, and demonstrates how these features can be used to aid recovery.

INDEX TERMS Btrfs, copy-on-write filesystems, deleted file recovery, extent-based recovery, filesystem forensics, metadata reconstruction

I. INTRODUCTION

B-tree File System (Btrfs) is a modern Linux filesystem designed to overcome scalability, reliability, and feature limitations of traditional Unix filesystems [1]. It integrates copy-on-write (CoW), snapshots, subvolumes, and data checksumming, along with built-in volume management, into a single design. As a result, Btrfs has seen widespread adoption and is used as the default or recommended filesystem in several major Linux distributions, as well as in production server environments, desktops, and embedded systems [2]. Consequently, Btrfs filesystem images increasingly appear in incident response and digital forensic investigations.

Effective forensic analysis requires reliable reconstruction of file data, file paths, and metadata such as creation and modification timestamps [3]. As Btrfs adoption grows, the lack of robust forensic recovery techniques becomes a scientific and practical problem.

Btrfs poses significant challenges to forensic analysis due to its fundamental design choices. Unlike traditional filesystems that rely on fixed metadata structures such as inode tables and block bitmaps, Btrfs stores nearly all metadata in a collection of B-trees [1]. Updates never occur in place. However, new tree nodes are written and linked from new roots. As a result, many versions of file data may coexist within a single filesystem image. Additionally, Btrfs supports multiple subvolumes, each represented by an independent filesystem tree, meaning that a single image may contain multiple logical filesystem roots with coinciding object identifiers. These properties challenge many assumptions used by existing recovery and forensic techniques, which generally rely on centralized metadata locations, linear allocation maps, or journal-based artifacts.

In this paper, we present a recovery algorithm that operates directly on a raw Btrfs filesystem image by fully parsing

its on-disk structures. At a high level, the algorithm moves as follows. First, the active filesystem tree is traversed to enumerate all present files. Next, the image is analyzed to locate and reconstruct older filesystem tree roots that are no longer referenced by the active root tree. These historical trees are reconstructed and traversed to identify filesystem objects that are absent from the current active root tree, allowing files to be classified as deleted or undeleted. For each discovered file object, the algorithm uses the object identifier to locate the related inode information and extent data, enabling reconstruction of file contents, paths, and metadata.

Compared to existing Btrfs recovery approaches, the proposed algorithm is assessed using factual recovery metrics, including recovery precision, integrity verifiability, file path, and metadata reconstruction accuracy, and scale as the size of the forensic evidence increases. Native Btrfs utilities rely on file extraction methods that fail to distinguish between deleted and non-deleted files, which is a critical detail in forensic investigations. They also fail to recover key details such as file paths and metadata. Most commercial signature-based carving tools achieve higher recall for data but suffer from low precision, lack of path and inode information extraction, inability to reassemble fragmented or compressed extents, and lack of filesystem-consistent integrity validation [4] [5]. Prior forensic approaches based on orphan-item or extent extraction improve recall over carving from Btrfs tree items, but exhibit low accuracy because the edge case they rely on is rarely encountered.

Section II (Background) presents the necessary details on Btrfs on-disk design relevant to forensic recovery. It explains the role of the superblock, root tree, filesystem trees, chunk tree, and extent metadata, and details how CoW updates preserve historical filesystem states. This section establishes the key detail that old filesystem roots and tree nodes may remain physically present in a valid filesystem image and can be leveraged for recovery.

Section III (Related Work) surveys existing Btrfs recovery and forensic approaches, including native Btrfs utilities, signature-based carving tools, commercial recovery software, and prior academic work. It highlights what was learned from them, the limitations of these approaches, particularly their reliance on heuristics, orphan items, and motivates the need for a recovery system that prioritizes all key file characteristics.

Section IV (Problem Definition) formally defines the scope of the recovery problem addressed in the paper. It specifies the required inputs, the expected outputs, and the assumptions under which the method operates. This section also states explicit constraints and limitations.

Section V (Method Overview) provides a high-level description of the proposed recovery method. It outlines the overall workflow. This section explains how deleted file candidates are identified through object identifier comparison rather than heuristics.

Section VI (Algorithm Detail) describes the recovery

algorithm in detail. It presents the individual stages of the method and their algorithms.

Section VII (Forensic Soundness Considerations) describes how the proposed implementation interacts with the input image to perform recovery operations without breaching basic forensic protocol related to data handling. It covers how the implementation handles logging and false-positives to provide adequate details needed for a structured forensic report and support the chain-of-custody.

Section VIII (Results) shows how the implementation performs in its primary use case, which is forensic recovery from OS dumps. This section tries to replicate a realistic scenario, where files are deleted in multiple deletion cycles, and testing recovery across them helps introduce edge cases with respect to the recovery process and its failure modes.

Section IX (Expanded Experimental Evaluation) covers more cases that are required to address the recovery capabilities of the tool. All specific cases are tested separately to provide the best understanding of the tool's behavior under them individually.

Section X (Limitations and Scope) discusses inherent limitations of the proposed method. It explains scenarios in which recovery is not possible. This section also discusses potential extensions, such as deep leaf scanning, and clarifies the intended forensic scope and the approach's applicability constraints.

Section XI (Conclusion) summarizes the contributions of the proposed method, reiterates the structural constraints under which deterministic recovery holds, and identifies directions for future work.

II. BACKGROUND

Btrfs stores filesystem state using a set of CoW B-trees rather than fixed metadata tables [1]. One of the few structures with a fixed physical location is the superblock, which contains information needed to locate the root tree. The root tree serves as an index of all other trees in the filesystem, such as the filesystem tree, the extent tree, and the checksum tree [1].

Each Btrfs subvolume is represented by its own filesystem tree. In this work, analysis is intentionally restricted to the filesystem trees and the metadata items they reference, specifically directory-related items used to reconstruct paths, inode items used to recover file metadata, and extent data items used to locate file contents [6]. File reconstruction in Btrfs requires correlating information distributed across multiple item types within a filesystem tree. Directory and reference items provide file names and parent-child relationships, enabling the reconstruction of full paths.

Btrfs items are referenced by logical addresses, which are mapped to physical disk locations via the chunk tree [1]. The complete on-disk layout of these structures, including the superblock, node headers, and item types, is formally documented in the Btrfs on-disk format specification [7]. This means that recovery from a filesystem image requires explicit address translation rather than direct offset interpre-

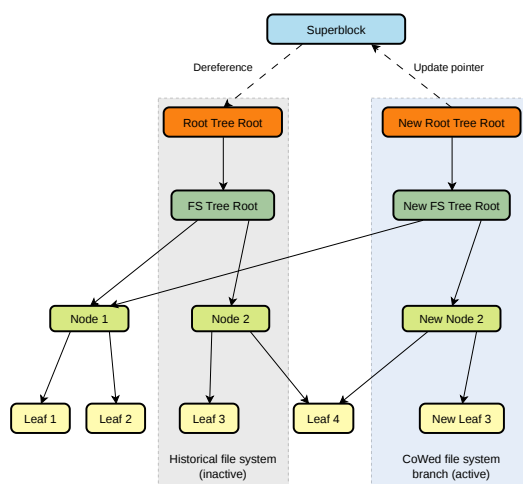


FIGURE 1. Effect of CoW on the file system

tation. Logical-to-physical translation is therefore required to access any Btrfs item during analysis.

A defining property of Btrfs is its use of CoW updates [1] [8]. The theoretical basis for this design was formally introduced by Rodeh [9] and later integrated into the Btrfs design [1]. A leaf of the B-tree is never modified in place. However, updates create new versions of tree nodes, and new root nodes are written to reference the updated subtrees. As a result, older versions of filesystem trees may remain physically present in the filesystem image even after they are no longer referenced by the new root tree, as shown in Fig. 1 [10] [8]. The structure and evolution of Btrfs trees under CoW updates and deletion have been described in prior forensic analysis work, most notably by Juch, who provides a detailed examination of Btrfs on-disk structures and their implications for forensic analysis. In Btrfs, updating a leaf or node triggers a ripple of CoW updates up to the root node, which, to our advantage, leaves the outdated roots of the filesystem in the image for later reconstruction [10]. This observation underpins the recovery approach presented in this paper, which treats historical filesystem trees as first-class sources of recoverable state rather than relying on isolated artifacts or heuristics [10].

III. RELATED WORK

Digital forensic reconstruction spans several substrates that share the common goals of verifiable integrity and provenance attribution. At the media-content layer, which includes detecting post-processing and tampering in images [11], distinguishing authentic from synthetically generated video [12], and embedding recoverable provenance and leakage-tracing information through robust watermarking [13], [14]. The present work pursues the analogous objectives, which are the bit-exact integrity verification and reconstruction of file origin at the filesystem layer, where the relevant evidence

is not perceptual content but on-disk metadata structure. Within storage forensics specifically, Btrfs differs from other filesystems in that deletion removes references rather than data, and historical filesystem states may persist due to copy-on-write.

The filesystem architecture and allocation mapping layer, including extent and chunk trees, were formally described by Rodeh et al. [1]. These properties complicate forensic reconstruction because the presence of metadata does not necessarily signify the recoverability of file contents.

A. BTRFS STRUCTURE-ORIENTED FORENSIC ANALYSIS

Juch conducted one of the first forensic analyses of Btrfs's internal structures. He defined six forensic artifacts, which are filesystem metadata, filesystem structure, file metadata, allocation status, contents of allocated files, and data from unallocated files. His results show that the first five artifacts can be extracted directly by parsing the active Btrfs B-tree metadata structures. However, recovering deleted content cannot be guaranteed solely through structures, because deletion removes metadata references to the underlying extents. Therefore, recovery of deleted file data must primarily rely on heuristic methods, such as scanning unallocated space [10]. Wani and Bhat analyzed the lifecycle of forensic artifacts and the dynamic behavior of the Btrfs filesystem. They focused on filesystem operations such as deletion, node balancing, and aging that affect the persistence of evidence within B-tree structures. As Btrfs uses CoW, deleted artifacts may persist in older leaf nodes or in unreferenced extents rather than being overwritten immediately. Their research also examined the role of orphan items. When a B-tree deletion results in node merging, the filesystem creates a new node using the CoW mechanism and fills it with items from neighboring nodes. In the process, most metadata related to the deleted file is overwritten. But small fragments of deleted metadata, including orphan items, may remain in the newly created node if the degree of merging is limited. Therefore, the original pre-merge node becomes the primary source for potential forensic recovery of deleted artifacts [15].

Their analysis also identified a limitation in recovery for very small files. Btrfs stores files smaller than roughly 4 KB as inline extents inside metadata leaf nodes. When such files are deleted, the corresponding metadata node is updated, and succeeding B-tree balancing operations can overwrite the node containing the embedded data. Due to this, small inline files have a much lower probability of forensic recovery [15]. The anti-forensic implications of Btrfs's design have also been studied. Wani et al. demonstrated that Btrfs's CoW mechanism makes secure file wiping difficult and that residual forensic evidence persists in ways unique to the filesystem's architecture [16], further motivating the need for native-structure-aware recovery.

These works show where evidence may reside, but do not specify how complete files can be reconstructed with integrity verification.

B. METADATA AND ARTIFACT EXTRACTION APPROACHES

Subsequent work proposed recovery procedures based on directory entries, extent records, and orphan items [17]. Such approaches increase the likelihood of locating residual data but rely on heuristic interpretation of metadata presence. Recovered outputs are therefore investigative artifacts rather than verifiable reconstructions [17], since bit-exact correctness is not established.

Prior forensic analysis of other Linux filesystems, such as Ext4, has similarly demonstrated that inode-level metadata parsing yields higher reconstruction fidelity than carving alone [18], and that inode carving specifically fails to recover directory hierarchies [19]

This fundamental limitation stems from Btrfs's CoW architecture. While metadata artifacts, such as directory entries and orphan items, may be stored within disconnected B-tree nodes during routine node balancing, the physical data extents they reference are flagged as unallocated upon deletion and are not overwritten immediately. However, this same CoW mechanism can be leveraged as a forensic advantage, leaving a structural trail of historical filesystem leaves and nodes. The current necessity in Btrfs forensics is to adapt to CoW. By parsing these historical Btrfs artifacts with strict precision, it is possible to reconstruct the exact state of the file system prior to data reallocation. Our proposed methodology achieves this, shifting from heuristic approximation to a structured recovery model that leverages Btrfs's internal CoW mechanics directly.

C. TOOL SUPPORT

The tool support began with the foundational single-device [10] work by Andreas Juch, which implemented static parsing. He built support for The Sleuthkit (TSK) [20], a widely used toolkit when it comes to open source file system forensics [21], that allowed it to locate the Btrfs Superblock, parse the Root Tree, and walk down the B-trees to extract standard files. Because of the heuristic limitation, TSK was successfully programmed to reliably extract five of its six artifacts using exact data structures.

Hilgert formally introduced a new mandatory step, "Pool Analysis," which must occur before the file system analysis [22] [23]. Their earlier work extended Carrier's model to cover pooled storage file systems in general, first demonstrated on ZFS [23], with the BTRFS implementation following in subsequent work [22]. In a multi-device Btrfs setup, a file might be split across three physical hard drives. Hilgert added TSK support to extract the System Chunk Items from the Superblock, build the Chunk Tree, and seamlessly translate a single logical address into multiple physical addresses across different disks.

`btrfs restore` [24] is a native `btrfs-progs` utility that occupies a middle ground between strict tree parsing and content carving. It is a salvage tool that traverses Btrfs trees while relaxing the consistency checks enforced at mount time, attempting to skip metadata it cannot read and continue

extracting data from the remainder of the filesystem. It reads the root tree and the filesystem (subvolume) trees, using the chunk tree to translate logical addresses to physical offsets, and copies file content reconstructed from the extent data items it encounters. The operator must supply a historical tree-root address through the `-t` option, typically obtained from `btrfs-find-root`, or enumerate candidate roots with `-l` for recovery from historical roots.

Several properties make `btrfs restore` unsuitable as a forensic recovery method, although it remains a practical salvage tool. It performs no deleted-versus-present classification. When directed at a historical root, every file reachable from that root is copied indiscriminately, with no determination of which objects were deleted relative to the current filesystem state. It uses regex to look for files referenced by the tree it scans, instead of basing search or classification with respect to file object identifiers. Recovery is consequently scoped to whichever root the operator selects rather than evaluated systematically across all historical generations. While `btrfs restore` does reconstruct the directory hierarchy and file paths of the data it copies, it does not restore inode metadata unless the `-m` option is specified (with extended attributes and symbolic links requiring `-x` and `-S` respectively), so timeline-relevant metadata is absent under default invocation. Because its output depends on the supplied root and the chosen option set rather than being a single deterministic function of the image, and because completeness and provenance are not guaranteed, it does not satisfy the determinism, metadata-preservation, and provenance requirements expected of forensic reconstruction.

While enabling investigation, these tools enumerate filesystem structures and do not evaluate the reliability limits of deleted-file reconstruction or automate the recovery.

D. LIMITATIONS OF EXISTING APPROACHES

Prior Btrfs forensic research assumes that surviving metadata implies recoverability. However, because logical addresses in Btrfs require logical-to-physical translation via the chunk tree, metadata persistence alone does not guarantee that referenced data can still be resolved after remapping [15]. Existing approaches identify candidate artifacts but cannot determine whether reconstruction remains forensically valid. Signature-based carving tools, which reconstruct files from their content rather than metadata [25], achieve higher recall but suffer from low precision, inability to reassemble fragmented extents, and no filesystem-consistent integrity validation.

E. CONTRIBUTION OF THIS WORK

This work evaluates deleted-file recovery as a deterministic forensic reconstruction problem rather than an artifact discovery problem. The method parses raw filesystem structures, reconstructs files across subvolumes with complete file content and metadata recovery, and validates results using cryptographic identity instead of format interpretation or carving.

IV. PROBLEM DEFINITION

This work addresses the problem of recovering deleted files from a valid Btrfs file system image using metadata-driven file system parsing. The objective is to reconstruct deleted files while preserving file completeness, original directory hierarchy, and inode metadata for forensic analysis. The proposed method operates on an intact file system state and does not attempt to repair, restore, or reconstruct damaged or corrupted file systems.

In this work, deterministic recovery refers to a process whose output is fully determined by the on-disk structure of the input image. Each recovered file is produced by traversing native Btrfs metadata and reconstructed from extent items bounded by the inode-declared size. The method makes no recovery decisions based on file signatures, fragmentation heuristics, or partial-content inference. Failures to recover are structural conditions detectable at analysis time (chunk-tree turnover, ownership discontinuity, extent overwrite) and are reported explicitly.

Most practical forensic solutions depend on content-based file carving rather than filesystem structure-specific parsing, leading to the loss of filenames, directory hierarchies, and timestamps, all of which seriously reduce forensic value [20][3]. Furthermore, current forensic frameworks provide limited or experimental support for Btrfs deleted-file recovery and often rely on corrupted metadata analysis or heuristic reconstruction, producing difficult-to-verify, non-deterministic results that are unsuitable for use as forensic evidence.

A. OPERATIONAL SCOPE AND ASSUMPTIONS

1) Inputs

The method requires as input a static, raw disk image (.img) containing a valid and internally consistent Btrfs file system. The image must include readable superblock information and traversable Btrfs metadata trees.

Forensic container formats such as E01 are not supported directly and must be converted to a raw image format prior to analysis.

2) Outputs

The method produces an output directory containing recovered files, including deleted files and, optionally, non-deleted files when required for forensic completeness. Recovered files are organized in a directory structure corresponding to their original file paths within the file system.

Associated inode metadata is preserved and written to the recovered files, including creation time, inode change time, modification time, and access time where available. Only complete files reconstructed through metadata-based extent resolution are produced.

3) Assumptions

The method assumes that the disk image represents a non-live, immutable snapshot of the file system. The file system is assumed to be valid, internally consistent, and compliant

with documented Btrfs on-disk structures. File system-level encryption is assumed not to be in use.

4) Constraints and Limitations

Recovery from solid-state drive (SSD)-backed filesystem images introduces additional complexity, as garbage collection operations may purge unallocated extents independently of filesystem-level deletion [26]. The proposed method operates on acquired static images and is therefore independent of storage medium behavior during analysis.

The method is limited to raw .img file system images. Recovery is possible only when the inode and extent metadata associated with deleted files remain present in B-tree leaf nodes. If the metadata leaf nodes referencing a deleted file have been overwritten, logical recovery through file system parsing is not feasible.

The method relies exclusively on metadata-driven parsing and explicitly excludes signature-based carving and heuristic content inference. When a file's extents cannot be fully resolved through the current chunk tree, content gaps are not filled by carving or pattern matching. Instead, the reconstruction outcome is classified among one of the tool's failure cases. Advanced techniques such as deep leaf scanning or historical chunk-tree reconstruction are not implemented in the current version.

B. DELETION CLASSIFICATION PREDICATE

Let G_c denote the current filesystem generation as recorded in the superblock. A regular file f is classified as deleted with respect to subvolume s if and only if there exists a generation $g < G_c$ such that f 's object identifier appears in s 's filesystem tree at generation g , and f 's object identifier is absent from s 's filesystem tree at generation G_c . Classification is performed independently per subvolume, and object identifiers from different subvolumes are never compared.

V. METHOD OVERVIEW

The presented method recovers deleted files from a valid Btrfs filesystem image by analyzing the current and historical filesystem trees and their leaves preserved through CoW. The approach relies solely on parsing logical filesystem metadata and does not perform file carving or filesystem repair, as shown in Fig. 2.

The method begins by parsing and validating the Btrfs superblock to obtain the current filesystem generation and locate the root tree. The chunk tree is then bootstrapped to enable logical-to-physical address translation, which is required for accessing all subsequent Btrfs metadata structures. The root tree is traversed to identify filesystem roots corresponding to the default filesystem and any additional subvolumes. These filesystem roots define separate filesystem namespaces. The recovery process is performed separately for each filesystem root to avoid object identifier collisions across subvolumes.

For a given filesystem root, the corresponding filesystem tree at the current generation is traversed using depth-first

search until leaf nodes are reached. During this traversal, Btrfs directory items are examined to identify present files. Each directory item provides a file name and its object identifier, which are used to construct the set of file object identifiers for undeleted files in the current state of the filesystem. To identify deleted files for the same filesystem root, the method scans the filesystem image for historical root tree blocks with generation numbers lower than the current generation. For each historical root tree identified, the filesystem root belonging to the currently examined filesystem or subvolume is located. These historical filesystem roots represent earlier versions of the same filesystem namespace, preserved by CoW.

Each historical filesystem tree is then traversed to extract directory items and construct a set of historical file object identifiers. Deleted file candidates are defined as object identifiers that appear in the historical filesystem tree but are absent from the current filesystem object identifier set. For each deleted file candidate, inode and extent items associated with the corresponding object identifier are collected for reconstruction. Directory items in Btrfs do not store explicit parent path information, but the implementation tracks parent object identifiers during traversal to reconstruct the directory path. File contents are reconstructed from extent items associated with the object ID, using offset attributes to reassemble the file data in the correct order.

The process is repeated independently for each filesystem root and subvolume, with all intermediate states cleared between iterations.

VI. ALGORITHM DETAIL

A. SYSTEM WORKFLOW

The recovery procedure is organized as a single deterministic driver that invokes the stage-specific algorithms (Algorithms 2–8) in a fixed order. The driver, given in Algorithm 1, separates a one-time initialization and discovery phase from a per-subvolume recovery loop. This subsection describes the control flow and states explicitly when each algorithm is invoked. The initialization phase executes once per image. The superblock is parsed and validated, the current filesystem generation G_c is extracted, and the chunk tree is bootstrapped to provide logical-to-physical address translation. The root tree is then traversed to enumerate the default filesystem root and all subvolume roots, using the same depth-first procedure as Algorithm 2. Finally, the image is scanned for historical root tree blocks, which Algorithm 3 indexes by generation into a map of root trees. If the superblock is invalid or no filesystem roots can be resolved, the driver terminates without recovery. The recovery loop then executes once per subvolume, with all intermediate states reset between iterations so that object identifiers from different subvolumes are never compared. Each iteration performs three passes over the same traversal engine. In Pass A (present-file identification), the current-generation filesystem tree of the subvolume is traversed with Algorithm 2, and each leaf is processed by Algorithm 5 to collect inode and extent items and by Algorithm 6 to build

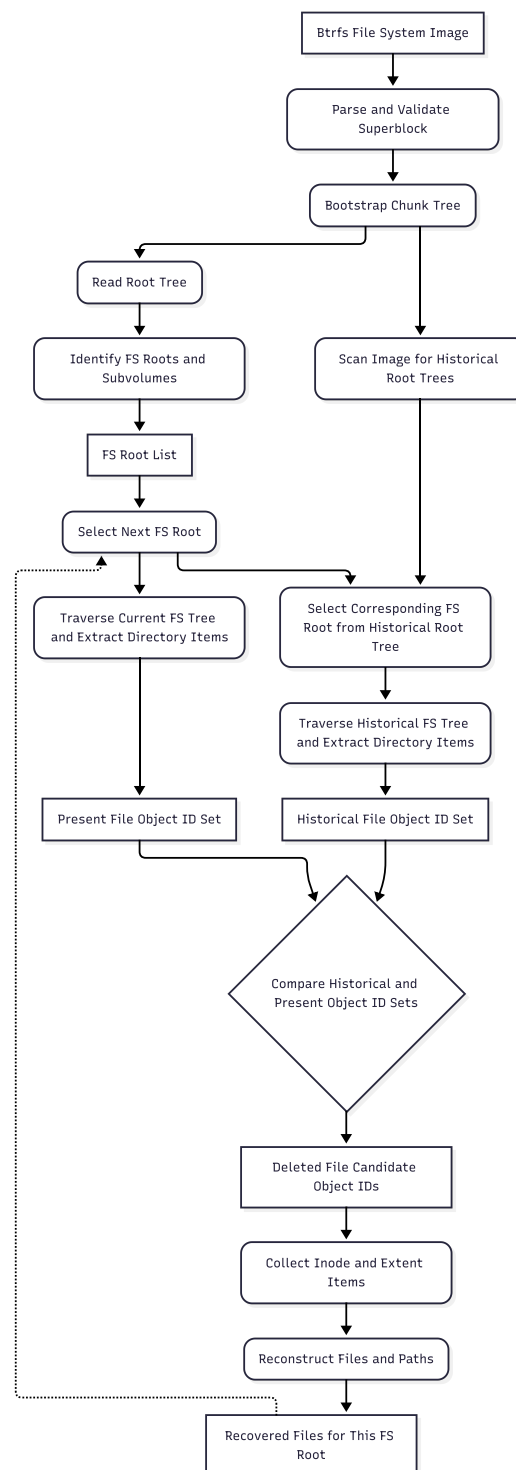


FIGURE 2. Method Overview to Recover Deleted Files

the set of present object identifiers and the parent-directory records. In Pass B (deleted-file identification), Algorithm 4 selects, for each historical generation in descending order, the root owned by the current subvolume and traverses it with Algorithm 2. Each leaf is again processed by Algorithm 5

and by Algorithm 7, which marks as deleted candidates those object identifiers that appear in the historical tree but are absent from the present set established in Pass A. In Pass C (reconstruction), each deleted candidate has its path resolved by parent-identifier recursion and its content reconstructed by Algorithm 8, which bounds reconstruction by the inode-declared size and assigns an outcome label.

Algorithm 1 Recovery Driver (Orchestration)

Require: Raw Btrfs image I

Ensure: Reconstructed deleted files with paths, metadata, and outcome labels

```

1:  $(sb, valid) \leftarrow \text{PARSESUPERBLOCK}(I)$ 
2: if  $\neg valid$  then
3:   return  $\triangleright$  not a valid Btrfs image
4: end if
5:  $G_c \leftarrow sb.generation$ 
6:  $ChunkTree \leftarrow \text{BOOTSTRAPCHUNKTREE}(sb)$   $\triangleright$ 
   logical  $\rightarrow$  physical
7:  $Roots \leftarrow \text{ENUMERATEFSROOTS}(sb, ChunkTree)$   $\triangleright$ 
   Alg. 2 on root tree
8: if  $Roots = \emptyset$  then
9:   return
10: end if
11:  $RootTreeMap \leftarrow \text{DISCOVERHISTORICALROOTS}(G_c)$ 
    $\triangleright$  Alg. 3
12: for each subvolume root  $s \in Roots$  do
13:   reset  $PresentObjectIDs$ ,  $DeletedCandidateIDs$ ,
14:    $InodeItemSet$ ,  $ExtentItemSet$ ,
15:    $VisitedNodes$ ,  $ParentMap$ 
   // Pass A — present-file identification
16:   TRAVERSENODE( $s.currentRoot$ )  $\triangleright$  Alg. 2, present
   mode
   // each leaf: Alg. 5, then Alg. 6
   // Pass B — deleted-file identification
17:   PROCESSHISTORICAL-
   TREES( $RootTreeMap, s.id$ )  $\triangleright$  Alg. 4  $\rightarrow$  Alg. 2,
   deleted mode
   // each leaf: Alg. 5, then Alg. 7
   // Pass C — reconstruction
18:   for each  $oid \in DeletedCandidateIDs$  do
19:      $path \leftarrow \text{RECONSTRUCTPATH}(oid, ParentMap)$ 
20:      $(file, outcome) \leftarrow$ 
       RECONSTRUCTFILE( $oid, InodeItemSet$ ,
21:        $ExtentItemSet, ChunkTree$ )  $\triangleright$  Alg. 8
22:     emit  $(path, file, metadata, outcome)$ 
23:   end for
24: end for

```

B. SUPERBLOCK PARSING AND INITIALIZATION

The algorithm begins by locating and parsing the primary Btrfs superblock from the filesystem image. The superblock is used to verify that the image contains a valid Btrfs filesystem using the magic number and to extract the current gen-

eration number, which represents the most recent committed transaction.

The generation number obtained from the superblock is later used to distinguish the current filesystem state from historical filesystem trees preserved due to CoW semantics.

To enable access to Btrfs metadata structures stored at logical addresses, the chunk tree is initialized using information from the superblock. Chunk tree initialization is treated as a prerequisite step that provides logical-to-physical address translation for subsequent metadata traversal. The internal operation of the chunk tree bootstrap process is not modified or extended by the proposed method.

If the superblock cannot be read or fails basic consistency checks, the algorithm terminates without attempting further recovery.

C. FILESYSTEM ROOT AND SUBVOLUME ROOT IDENTIFICATION

After initialization, the algorithm identifies the filesystem tree roots that belong to the current filesystem state. In Btrfs, the locations of filesystem trees are stored as items within the root tree. These items reference the roots of the default filesystem and additional filesystem trees associated with subvolumes.

The root node is located using information obtained from the superblock and accessed through logical-to-physical address translation enabled by the chunk tree. The root tree is a general Btrfs B-tree and may have one or more levels of internal nodes, depending on the number of subvolumes, snapshots, and other roots present in the filesystem. It is therefore traversed using the same depth-first procedure described in Algorithm 2.

The root tree is parsed to locate root items. The default filesystem root is identified by its default object identifier, while additional filesystem roots belonging to subvolumes are identified by their object identifiers and associated root reference items. For each identified filesystem root item, the logical address of the corresponding filesystem tree root is extracted and translated to a physical offset. Each filesystem tree root is then read into memory and added to a collection of filesystem roots. This collection includes the default filesystem root and all additional filesystem roots associated with subvolumes. These filesystem roots serve as entry points for later filesystem traversal.

Filesystem roots that cannot be mapped through the chunk tree or whose headers cannot be parsed correctly are excluded, and the algorithm terminates if no valid filesystem roots can be identified.

D. FILESYSTEM TREE TRAVERSAL

For each filesystem root identified in the previous stage, the affiliated filesystem tree is traversed to extract directory items, inode items, and extent items needed for recovery. Traversal is performed recursively from the root of the filesystem tree until leaf nodes are reached.

At each node, the Btrfs header is parsed to determine the node level. Internal nodes are processed by resolving child block pointers and recursively traversing each referenced child node. Leaf nodes are processed to extract relevant Btrfs items, including directory, inode, and extent items.

To avoid traversing blocks belonging to unrelated filesystem trees or obsolete copy-on-write versions, child nodes are validated against the parent node pointer metadata before traversal. Specifically, the child generation, logical address, and tree level must match the values recorded in the parent pointer. Nodes failing these consistency checks are treated as overwritten or reallocated blocks and are pruned from traversal. Nodes that do not match the generation number, or their logical address referenced by their parent, and are not one level below them, are skipped.

A key part of the traversal strategy is node-level caching to avoid redundant processing. Because Btrfs uses CoW, filesystem trees across different generations often share the same internal and leaf nodes. When multiple filesystem roots reference the same logical node, straightforward traversal would repeatedly visit and process the same node, resulting in significant performance degradation.

To address this, the algorithm maintains a cache of visited node logical addresses. Before traversing a node, its logical address is checked against this cache. If the node has already been visited, traversal of that subtree is skipped. This optimization prevents redundant processing of shared nodes and significantly reduces traversal time when analyzing historical filesystem trees.

Leaf nodes are processed only once per traversal context, and all extracted items are registered accordingly. Without this node-caching mechanism, traversing historical filesystem trees can result in repeated processing of identical nodes or leaves, leading to impractical execution times for large filesystems, especially image dumps of operating systems, where deletions occur frequently, increasing the number of historical filesystems generated with minimal leaf or node changes.

Let the filesystem tree of a single subvolume have n leaves, branching factor f , and height $h = \lceil \log_f n \rceil$. Its total node count is

$$N = \sum_{i=0}^h \left\lceil \frac{n}{f^i} \right\rceil < \frac{f}{f-1} n = \Theta(n) \quad (1)$$

Recovery traverses G generations of this tree (the current generation together with the $G-1$ historical roots discovered in Algorithm 3).

a: Traversal without caching.

Re-walking each generation independently—as a per-root invocation of a salvage tool does—processes

$$T_{\text{naive}} = \sum_{g=1}^G N_g \approx GN = \Theta(Gn) \quad (2)$$

b: CoW sharing.

Between two committed generations, every modified leaf forces a copy-on-write of its entire root-to-leaf path; all sibling subtrees retain their original logical addresses and are physically shared. If m_g leaves change at generation g , the number of newly written nodes is bounded by the length of those ripple paths,

$$\delta_g \leq m_g(h+1) \quad (3)$$

while the remaining $N - \delta_g$ nodes are identical blocks already present in an earlier generation.

c: Traversal with the VisitedNodes cache.

Because the cache is keyed on logical node address, each CoW-shared block is admitted at most once. The work, therefore, equals the size of the union of visited sets (V), not their sum:

$$T_{\text{cache}} = \left| \bigcup_{g=1}^G V_g \right| = N + \sum_{g=2}^G \delta_g \approx N + (G-1)\bar{\delta} \quad (4)$$

where $\bar{\delta}$ is the mean per-generation churn.

d: Speedup.

The ratio of the two costs is

$$S = \frac{T_{\text{naive}}}{T_{\text{cache}}} = \frac{GN}{N + (G-1)\bar{\delta}} \quad \lim_{G \rightarrow \infty} S = \frac{N}{\bar{\delta}} \quad (5)$$

Substituting $\bar{\delta} \leq \bar{m}(h+1) = O(\bar{m} \log_f n)$ and $N = \Theta(n)$ gives

$$T_{\text{cache}} = O(n + G\bar{m} \log_f n) \quad S = \Theta\left(\frac{n}{\bar{m} \log_f n}\right). \quad (6)$$

Algorithm 2 defines only the traversal and pruning strategy. Extraction of filesystem metadata is performed during leaf processing and is described separately.

E. HISTORICAL ROOT TREE DISCOVERY AND PROCESSING

1) Historical Root Tree Discovery

In Btrfs, historical filesystem trees are preserved due to CoW, but historical root trees are not directly referenced from the current filesystem metadata. Therefore, recovery of deleted files requires explicit discovery of historical root tree blocks from the filesystem image.

To locate historical root trees, the method scans filesystem regions mapped to Btrfs tree blocks using the chunk tree. Each candidate block is read and examined by parsing its Btrfs node header. A block is identified as a root tree node only if the ownership field in the header indicates root tree ownership. Blocks that do not satisfy this condition are discarded.

For each identified root tree node, the generation number recorded in the header is compared against the current filesystem generation obtained from the superblock. Root tree nodes whose generation is strictly less than the current filesystem

Algorithm 2 Filesystem Tree Traversal with Generation Val-
idation

Require: Filesystem tree root node r , chunk tree mapping
Ensure: Traverse only valid filesystem tree nodes and prune
 overwritten/reallocated blocks

```

1: TRAVERSENODE( $r$ )
2: procedure TRAVERSENODE( $n$ )
3:   Parse Btrfs header from node  $n$ 
4:   if node level = 0 then ▷ Leaf node
5:     Resolve physical address of  $n$ 
6:     PROCESSLEAF( $n$ )
7:   else
8:     if logical address of  $n \notin VisitedNodes$  then
9:       Add logical address of  $n$  to  $VisitedNodes$ 
10:    end if
11:    Extract child pointers from  $n$ 
12:    for each child pointer  $p$  do
13:      if  $p.blockptr \in VisitedNodes$  then
14:        continue
15:      end if
16:      Resolve physical address of  $p.blockptr$  using
  chunk tree
17:      if mapping does not exist then
18:        error
19:      end if
20:      Read child node  $c$ 
21:      Parse child header
22:      if child generation  $\neq p.generation$  then
23:        continue
24:      end if
25:      if child logical address  $\neq p.blockptr$  then
26:        continue
27:      end if
28:      if child level  $\neq$  parent level  $-1$  then
29:        continue
30:      end if
31:      Add  $p.blockptr$  to  $VisitedNodes$ 
32:      TRAVERSENODE( $c$ )
33:    end for
34:  end if
35: end procedure

```

generation G_c are retained as historical roots. Because the root tree is itself a multi-level B-tree, a single historical root tree may contribute several blocks sharing the same owner and generation (one per internal or leaf node within that tree). To produce a single entry per generation, the discovery procedure retains, for each observed generation, the block with the highest level. The node with the highest level for a given generation is, by construction, the root of the corresponding tree, and all lower-level nodes are reachable from it via the standard traversal defined in Algorithm 2. These nodes are stored in a generation-indexed mapping that links filesystem generations with the corresponding historical root tree blocks. Because not all generations are guaranteed to be

Algorithm 3 Historical Root Tree Discovery and Storage

Require: Chunk tree mapping (fully bootstrapped), current
 filesystem generation G_c
Ensure: Mapping from generation numbers to historical
 root tree root blocks, for generations $< G_c$

```

1: Initialize empty map  $RootTreeMap$ 
2: for each candidate tree block  $b$  in mapped tree regions
  do
3:   Parse Btrfs header from  $b$ 
4:   if header.owner = 1 then ▷ Root tree owner
5:     if header.generation  $< G_c$  then
6:        $g \leftarrow$  header.generation
7:       if  $g \notin RootTreeMap$  or header.level  $>$ 
  level( $RootTreeMap[g]$ ) then
8:          $RootTreeMap[g] \leftarrow b$ 
9:       end if
10:    end if
11:  end if
12: end for

```

present, this mapping may be scattered. Nodes at generation G_c correspond to the current root tree and are handled by the active-filesystem traversal.

This discovery process produces a set of historical root tree blocks that can later be used to locate historical filesystem roots related to specific subvolumes. The complete discovery and storage procedure is formalized in Algorithm 3.

2) Subvolume-Scoped Processing of Historical Filesystem Trees

Historical root trees identified during the discovery phase may contain filesystem roots for multiple subvolumes. Because object identifiers may overlap across subvolumes, deleted file recovery must be scoped to a single filesystem subvolume at a time.

For each filesystem root identified at the current generation, historical root trees are processed in descending generation order. For each historical root tree, filesystem roots and subvolume roots are extracted. From these, only the filesystem root whose ownership matches the currently processed subvolume identifier is selected.

Once the corresponding historical filesystem root is identified, the associated filesystem tree is traversed using the CoW-aware traversal procedure described in Algorithm 2. Traversal is performed in deleted-file identification mode, and directory items encountered during traversal are compared against the set of present file object identifiers previously established for the same subvolume.

Processing proceeds iteratively across successively older generations until no historical root trees remain. This procedure ensures that deleted file recovery is performed independently for each subvolume and that historical filesystem states are examined in a consistent chronological order. The complete procedure is formalized in Algorithm 4.

Algorithm 4 Subvolume-Scoped Processing of Historical Filesystem Trees

Require: Root tree map *RootTreeMap*, subvolume identifier *s*
Ensure: Historical filesystem trees for subvolume *s* are traversed

```

1:  $g \leftarrow$  current filesystem generation
2: while there exists a generation  $g' < g$  in RootTreeMap
   do
3:   Select the largest generation  $g' < g$  from RootTreeMap
4:    $g \leftarrow g'$ 
5:    $r \leftarrow \text{RootTreeMap}[g]$ 
6:   Extract filesystem roots from root tree r
7:   for each filesystem root f in r do
8:     Parse Btrfs header from f
9:     if header.owner = s then
10:      Traverse filesystem tree rooted at f
11:      using Algorithm 2 in deleted-file mode
12:    end if
13:  end for
14: end while

```

Algorithm 5 Leaf Scan and Metadata Collection

Require: Leaf node *l*
Ensure: All inode and extent items in *l* are collected

```

1: Parse Btrfs leaf header from l
2: Extract all items from leaf l
3: for each item i in l do
4:   if i is an inode item then
5:     Add i to InodeItemSet
6:   else if i is an extent item then
7:     Add i to ExtentItemSet
8:   end if
9: end for

```

F. LEAF PROCESSING AND METADATA EXTRACTION

Leaf nodes in a Btrfs filesystem tree contain the actual filesystem metadata items [6] required for file enumeration and reconstruction. During filesystem tree traversal (Algorithm 2), every reachable leaf node is processed to extract directory items, inode items, and extent items. This is also done during traversal of the active filesystem if the forensic investigation requires extracting existing files.

Leaf processing is conceptually divided into two stages: metadata collection, which is common to all traversals, and directory item interpretation, which differs depending on whether the traversal is executed for initial identification of files present or the examination of deleted files in the historical filesystem tree.

1) Metadata Collection

For every leaf node encountered, all inode items and extent items contained in the leaf are collected, regardless of whether the active filesystem tree or the historical tree is

Algorithm 6 Present File Identification from Leaf Nodes

Require: Leaf node *l*, output set *PresentObjectIDs*
Ensure: Present file object identifiers are recorded

```

1: for each directory item d in leaf l do
2:   if d represents a directory then
3:     Record directory object identifier
4:     Record parent object identifier
5:   else if d represents a regular file then
6:     if d.objectid  $\notin$  PresentObjectIDs then
7:       Add d.objectid to PresentObjectIDs
8:     Record directory item d
9:   end if
10: end if
11: end for

```

being traversed. This behavior is formalized in Algorithm 5. Inode and extent items are not filtered based on file allocation status because directory, inode, and extent items associated with the same file object identifier may reside in different leaf nodes. Postponing filtering ensures that all metadata required for later file reconstruction is available when a deleted file candidate is identified.

This design choice is critical for correctness, as without global metadata collection, reconstruction may fail when directory entries and data extents are not present within the same leaf node.

2) Present File Identification

During traversal of the current-generation filesystem tree, directory items are interpreted to identify files that are not deleted. This phase constructs the set of present file object identifiers and records directory metadata (parent object identifier) needed for later path reconstruction. The procedure is defined in Algorithm 6.

For each directory item encountered, the item type is examined to distinguish between directories and regular files. Directory entries are recorded along with their object identifiers to enable reconstruction of directory paths. For regular file entries, the corresponding object identifier is added to the set of present file object identifiers if it has not been observed previously. This set represents the authoritative view of files present in the active filesystem tree.

3) Deleted File Identification

Traversal of historical filesystem trees is performed only after the set of present file object identifiers has been created. During this phase, directory items are interpreted to identify candidate deleted files, as formalized in Algorithm 7.

For each directory item processed in a historical filesystem tree, its object identifier is compared against the set of present file object identifiers. Regular file entries whose object identifiers are absent from the present set are considered as deleted files. Duplicate candidates are suppressed to avoid redundant processing. Directory entries are again recorded for later path reconstruction.

Algorithm 7 Deleted File Candidate Identification from Leaf Nodes

Require: Leaf node l , set of present file object identifiers
Ensure: Deleted file candidate object identifiers are recorded

```

1: for each directory item  $d$  in leaf  $l$  do
2:   if  $d$  represents a directory then
3:     Record directory object identifier
4:     Record parent object identifier
5:   else if  $d$  represents a regular file then
6:     if  $d.objectid \notin PresentObjectIDs$  then
7:       if  $d.objectid \notin DeletedCandidateIDs$  then
8:         Add  $d.objectid$  to  $DeletedCandidateIDs$ 
9:       Record directory item  $d$ 
10:    end if
11:  end if
12: end if
13: end for

```

This comparison-based approach ensures that deleted files are identified solely through logical filesystem tree analysis, without relying on file carving or similar recovery techniques.

4) File Path Reconstruction

Because Btrfs directory items do not store absolute paths, reconstructed files must have their original paths inferred from directory data collected during processing of leaves in the form of parent object identifiers. This is achieved by recursively resolving parent directory object identifiers until a root directory is reached. If the recursion cannot resolve to the subvolume root, for example, because an intermediate directory's parent record was not collected during traversal, reconstruction terminates at the deepest resolvable ancestor and emits a partial path.

5) Accurate File Reconstruction from Inodes and Extents

File reconstruction is performed using inode and extent data collected during filesystem tree traversal. Because inode and extent items may be spread across multiple filesystem tree leaves, reconstruction operates on globally collected metadata rather than relying on just its local leaf context.

For a given file object identifier, the corresponding inode item is selected uniquely from the inode metadata set. The inode provides the authoritative logical file size, which is used to bound reconstruction and ensure byte-accurate recovery. All extent items associated with the same object identifier are then selected from the extent metadata set. A file may be associated with multiple extents, each describing a specific range of file data.

Extent items are first ordered by their logical file offset to ensure correct reconstruction order. Ordering is performed using the offset attribute of each extent key, which represents the logical position of the extent within the file. This guar-

antees that extents are written in the same sequence as in the original file.

Some extents may reference data blocks that contain additional data preceding the actual file content. To handle this, the physical read offset for each extent is computed by combining the logical extent address with the extent-specific offset attribute. This ensures that only the relevant portion of each data block is used during reconstruction.

Reconstruction proceeds by seeking to each extent's file offset (the extent item's key offset) and writing its data there. Holes between extents, preallocated extents, and extents whose logical address cannot be mapped through the current chunk tree are left as unwritten regions. A final truncation to the inode-declared logical size materializes these regions as zero-filled bytes, which is consistent with POSIX sparse-file and prealloc semantics [27]. This guarantees that the reconstructed file size exactly matches the inode-declared size and that content placement preserves the original file layout, including holes, regardless of whether all extents resolve.

This inode-bounded reconstruction shown in Algorithm 8 approach avoids rampant trimming of trailing null bytes and ensures the theoretical correctness of recovered file contents, including preservation of file hashes for verification in controlled experiments. Because reconstruction is bounded strictly by inode file size, recovered files contain no extraneous padding bytes and preserve original file length and content ordering when extent data is intact.

G. CLASSIFICATION SEMANTICS AND EDGE CASES

The deletion classification predicate operates on Btrfs object identifiers. Its soundness depends on properties of object-identifier allocation and on the semantics of filesystem operations that change the set of present object identifiers. This section validates the predicate against each such Btrfs-supported operation and identifies the resulting behavior, including failure modes that arise when structural assumptions break down.

1) Object Identifier Monotonicity

The soundness of the deletion classification rests on the assumption that object identifiers are not reused after deletion. In current Btrfs implementations, object-identifier allocation is performed by `btrfs_get_free_objectid` in `fs/btrfs/disk-io.c` [28], which increments a per-root counter `free_objectid` stored in `struct btrfs_root` and persistent within a mount session. The counter is strictly monotonic, where a freed object identifier is not subsequently reissued. Earlier kernel versions implemented hole-scanning allocators that could potentially reuse identifiers, but these have been superseded. Modern Btrfs versions therefore guarantee monotonic allocation within each filesystem tree, ensuring that an object identifier observed in a historical tree always refers to the same logical filesystem object and never to a later distinct file. The set difference is therefore sound by construction within a single subvolume.

Algorithm 8 Extent-Accurate File Reconstruction

Require: Object id o , inode set, extent set, chunk map
Ensure: File bounded by inode size; outcome label

- 1: Select inode i for o ; select extents of o
- 2: Sort extents by file offset; open output
- 3: $resolved \leftarrow 0$; $unresolved \leftarrow 0$
- 4: **for** each extent e in sorted order **do**
- 5: $o_f \leftarrow e.file_offset$
- 6: $b \leftarrow \min(e.byte_count, i.size - o_f)$
- 7: **if** e is inline **then**
- 8: **if** $e.compression \neq 0$ **then**
- 9: $c \leftarrow \text{DECOMPRESS}(e.content, e.compression)$
- 10: **else**
- 11: $c \leftarrow e.content$
- 12: **end if**
- 13: Seek to 0; write c
- 14: **else if** $e.type = \text{PREALLOC}$ or $e.disk_bytentr = 0$ **then**
- 15: **continue** ▷ hole
- 16: **else if** $\text{MAP}(e.logical) = \perp$ **then**
- 17: Log $(o, e.logical, b)$ unresolved
- 18: $unresolved += 1$; **continue**
- 19: **else if** $e.compression \neq 0$ **then**
- 20: $p \leftarrow \text{MAP}(e.logical)$ ▷ no offset
- 21: $raw \leftarrow \text{read } e.disk_num_bytes \text{ at } p$
- 22: $D \leftarrow \text{DECOMPRESS}(raw, e.compression)$
- 23: $data \leftarrow D[e.offset : e.offset + b]$
- 24: Seek to o_f ; write $data$
- 25: $resolved += 1$
- 26: **else**
- 27: $p \leftarrow \text{MAP}(e.logical + e.offset)$
- 28: Seek to o_f ; write b bytes from p
- 29: $resolved += 1$
- 30: **end if**
- 31: **end for**
- 32: Truncate output to $i.size$; apply inode metadata
- 33: **if** $unresolved = 0$ **then**
- 34: $outcome \leftarrow \text{COMPLETE}$
- 35: **else if** $resolved = 0$ **then**
- 36: $outcome \leftarrow \text{METADATA-ONLY}$
- 37: **else**
- 38: $outcome \leftarrow \text{PARTIAL}$
- 39: **end if**

2) Subvolume Scoping

Object identifiers are scoped to a single filesystem tree, and Btrfs permits identical object identifiers across different subvolumes [1]. A set-difference comparison across subvolumes would conflate distinct files. The algorithm restricts comparison to within a single filesystem tree at a time.

3) Rename and Intra-Subvolume Move

A rename or move within a subvolume modifies the directory item and inode-reference entries associated with a file but does not alter its object identifier. The renamed file's object identifier, therefore, remains in the current filesystem tree's object identifier set, and the file is not classified as deleted. The historical directory item bearing the previous name is observed during traversal of older generations but is not recorded as a deletion candidate, because its object identifier matches a present file. The previous name is therefore implicitly discarded, which preserves classification correctness at the cost of losing the prior path. Recovery of historical names would require separate analysis of historical inode-reference entries and is out of scope.

4) Cross-Subvolume Move

Subvolumes in Btrfs are independent namespaces. A move from one subvolume to another is implemented as a copy followed by an unlink in the source. The source subvolume, therefore, observes a true deletion of the original object identifier, which the predicate correctly classifies. The destination subvolume contains a new file with a newly allocated object identifier and is unaffected by classification in any other subvolume.

5) Snapshots

A Btrfs snapshot is a writable subvolume that shares extents with its source at the moment of creation but maintains its own filesystem tree and object identifier space [1]. Per-subvolume scoping, therefore, handles snapshots without modification. A file present in a snapshot but absent from the source subvolume is not classified as deleted in the snapshot's scope, because the predicate is evaluated independently for each subvolume. Snapshots may themselves serve as recovery entry points by treating each snapshot's filesystem root identically to any other subvolume root.

6) Deleted Directories

The algorithm maintains a mapping of file object identifiers for classification, while directory object identifiers are recorded separately in a parent-lookup table for path reconstruction. Directories are not themselves treated as deletion candidates. When a directory is deleted, all regular files within it become absent from the current filesystem tree, and the predicate classifies each such file as deleted on its own merits. The deleted directory's own metadata is not reconstructed, but its directory item within its parent, if still present in any historical leaf, remains available to the path-reconstruction routine. Path reconstruction, therefore, traverses the deleted directory transparently, even though the directory itself is not recovered as an object.

7) Hard Links

In Btrfs, multiple hard links to a single file are represented by separate inode-reference entries within the inode's item cluster (one per parent directory) and by matching directory and directory-index entries within each linking directory [6]. Hard links cannot cross subvolume boundaries.

The current implementation reconstructs paths by recording, for each file object identifier, a single parent directory identifier and recursing upward. The recording step in Algorithms 6 and 7 is gated on whether the file's object identifier has already been added to the present (or deleted-candidate) set. As a consequence, only the first directory item encountered for a given object identifier is retained. For files with multiple hard links, all directory items after the first are silently dropped, and only one path per file is reconstructed. This behavior is correct under the deletion predicate but incomplete with respect to path enumeration. Decoupling the directory-item recording step from the set-membership check would allow the parent-lookup table to

retain all (parent, name) pairs for each object identifier, and path reconstruction would then emit one path per recorded pair. This extension is left as future work.

For partial hard-link deletions, in which one link is removed, but the inode persists, the predicate correctly does not classify the file as deleted, because the object identifier remains in the present set. The current method does not address recovering the name of the removed link.

8) Missing Parent in Path Reconstruction

Path reconstruction proceeds by recursively resolving parent object identifiers, beginning at the file's parent and continuing upward until the subvolume root is reached. If recursion encounters an object identifier for which no parent record exists in either the current or historical metadata observed by the algorithm, recursion terminates at the deepest resolvable ancestor. The resulting partial path begins at that ancestor rather than at the subvolume root. In the current implementation, partial paths are not distinguished syntactically from full paths in the output, and investigators must identify the disconnected segment from contextual evidence such as the depth of the recovered directory hierarchy. Marking partial paths explicitly is a straightforward extension that would improve forensic transparency and is noted as future work.

VII. FORENSIC SOUNDNESS CONSIDERATIONS

Forensic recovery requires more than the production of recovered file content. The analysis process must preserve the integrity of the source evidence, produce verifiable outputs, and integrate with established investigator workflows and case documentation requirements. This section describes the forensic soundness properties of the proposed method.

A. READ-ONLY ANALYSIS

The method accesses the filesystem image exclusively through read operations. The algorithm parses on-disk structures, including the superblock, chunk tree, root tree, filesystem trees, and historical root tree blocks, without modifying the image at any stage. All reconstructed file content and associated metadata are written to a separate output directory specified at invocation. The input image, therefore, remains bit-identical to its acquired state throughout the analysis, and any forensic hash computed prior to analysis remains valid after analysis completes.

B. CRYPTOGRAPHIC HASHING OF INPUT AND OUTPUTS

Forensic validation requires that both the source evidence and the recovered artifacts be independently verifiable. The method supports this through hashing at two levels. The input image is treated as immutable evidence. Prior to analysis, the investigator computes a SHA-256 digest of the raw image and records it in the case file. Because the method accesses the image exclusively through read operations, the post-analysis digest is identical to the pre-analysis digest, and this

equality can be verified by the investigator or an independent analyst at any point after the run.

Recovered files are verified by hashing after extraction. The current implementation writes each reconstructed file to the output directory at its inode-declared logical size. SHA-256 digests are computed externally over the resulting files and recorded alongside the audit log. Because reconstruction is deterministic and bounded strictly by the inode-declared size, the digest of a recovered file is reproducible across independent runs on the same image and serves as the verification primitive. Emission of per-file digests directly by the tool, as part of the structured report, is identified as future work.

C. DETERMINISTIC REPRODUCIBILITY

The algorithm is deterministic by construction. It contains no stochastic components, no dependence on host-system state, and no parallel execution paths that affect output. Given the same input image, repeated executions produce byte-identical recovered files, identical recovered metadata, and consistent identification of deleted and present files. This property is essential for forensic admissibility, as it allows an independent analyst to reproduce the original investigator's findings on the same evidence and obtain the same result.

D. AUDIT LOGGING

The implementation emits a human-readable analysis log documenting each stage of the recovery process. The log records superblock validation, chunk-tree bootstrap parameters, root-tree location and traversal, enumeration of subvolumes, discovery of historical root-tree blocks with their generations and physical offsets, identification of deleted-file candidates, and per-file reconstruction outcomes, including extent layout and final status. An excerpt of this log appears in Appendix A. The log functions as a retraceable audit trail of the analysis: any decision made by the algorithm on a given image can be reconstructed from the log without re-executing the tool.

E. CHAIN-OF-CUSTODY SUPPORT

Chain of custody is maintained by the investigator and is documented at the case level rather than at the tool level. The proposed method supports the investigator's chain-of-custody obligations through three properties described above: the input image is not modified during analysis, outputs are deterministic and reproducible by an independent analyst, and an audit log documents the analysis step. The method does not itself record custody metadata, such as operator identity or case identifier; these are recorded by the investigator's case-management system, which incorporates the method's outputs.

F. CONSERVATIVE TRAVERSAL AND FALSE-POSITIVE AVOIDANCE

The accuracy of deleted-file classification depends not only on what the method recovers but on its refusal to recover what it cannot attribute with certainty. Because CoW reuses

freed logical ranges, a historical node's child pointers may, in the current on-disk state, reference blocks overwritten by unrelated trees. The per-node generation, level, and block pointer reference check enforced during traversal (Algorithm 2) detects this condition and prunes the divergent branch, ensuring that every item admitted into the inode, extent, and directory sets is attributable to the nodes that were not relocated or overwritten.

Crucially, this guard does not trade recall for precision in practice. Under CoW, unchanged historical subtrees are shared and remain reachable from multiple historical root generations. The highest-level block retained per generation by Algorithm 3 means a node that is still legitimately present on disk is, in general, re-reached through at least one historical root whose path to it has not been overwritten. Across all images in the evaluation, each node pruned via an ownership-mismatched pointer was subsequently re-reached intact via an alternate generation, yielding zero recoveries failed due to ownership discontinuity. The mechanism, therefore, improves precision, and no file was reconstructed from misattributed metadata at no observed cost to recall.

G. STRUCTURED FORENSIC REPORTING

The current implementation produces a human-readable analysis report suitable for an investigator's inspection. Integration with automated case-management pipelines would benefit from a machine-readable representation of the same information. Emission of a structured report alongside the human-readable log is identified as future work. The planned report would record per-file recovery outcomes, including object identifier, reconstructed path, logical size, inode timestamps, extent layout (inline, single-extent, or multi-extent), SHA-256 hash of the reconstructed content, recovery status (complete, overwritten, or partially recovered), and any detected failure condition.

VIII. RESULTS

The evaluation presented in this work was performed on a controlled dataset constructed specifically for this purpose, as no public Btrfs forensic corpora providing raw filesystem images suitable for tool evaluation exist at the time of writing. Standard forensic corpora such as NIST CFReDS and Digital Corpora do not include Btrfs images. Consequently, evaluation was performed on a single filesystem image with intentionally deleted files spanning multiple deletion cycles, file types, and extent layouts. While this restricts the statistical generalizability of the reported metrics, it represents the current ceiling of reproducible evaluation in this space.

A. DATASET CHARACTERISTICS

A total of 23 files were created in a Btrfs user subvolume. Nine files were deleted in different deletion cycles.

Files of different extensions were created (e.g., mp4, wav, txt, bin, jpg). File contents consisted exclusively of either deterministic ASCII patterns or pseudorandom data generated from /dev/urandom. Therefore, recovery correctness was

TABLE I. Deletion Cycles

Image	Files Deleted
1	None (ground truth)
2	image5.jpg
3	filex.txt, video1.mkv
4	audio1.wav, audio2.mp3
5	image3.bmp, binary.bin
6	file1.txt, video2.mp4

strictly based on cryptographic identity with ground truth rather than the file type or format.

Images were collected before any deletion and after each deletion cycle.

B. DELETION TIMELINE AND RECOVERY

Table I shows the sequence of stages of deletion. The output of the recovery tool for this final image is provided in Appendix A, showing how the algorithm is applied practically. It should be noted that the appendix output is heavily curated and does not represent the full output of the tool, and only the most relevant portions are retained for clarity. After identifying the current files by traversing the active filesystem tree, historical roots are located, and the filesystem tree of the target subvolume is traversed across these older generations. Files absent from the active tree but present in historical roots are classified as deleted. The results include deleted file entries with related attributes, including file paths, sizes, timestamps, and extent information (inline, single-extent, and multi-extent).

The algorithm was executed on the sixth filesystem image (see Table I), and the observed recovery statistics are consistent with those reported in Table II.

This experiment was conducted exclusively on the home subvolume. The read and write operations were performed on SSDs, which contributed to the negligible I/O overhead observed during traversal and extraction. All interaction with the Btrfs structures was performed entirely through the on-disk format, without mounting the filesystem.

C. RECOVERY OUTCOME TAXONOMY

1) Failure causes

Non-complete outcomes arise from three distinct structural conditions, of which 2/3 are explicitly detected and reported by the algorithm.

Missing chunk mapping: The chunk tree contains no entry covering the logical address of an extent. This results from chunk-tree turnover under allocation pressure and produces Partial or Metadata-only outcomes depending on how many of the file's extents are affected.

Overwritten data block: The chunk tree successfully translates the extent's logical address, and a read at the resulting physical address succeeds, but the underlying block has been reused for a subsequent write. This condition is not detectable by the algorithm without external ground.

Ownership discontinuity: Traversal terminates if the historical parent's child pointer fails generation/logical/level

validation, and any files reachable only through the abandoned branch are not enumerated to maintain accuracy.

To enable uniform reporting and clear diagnosis of failure conditions, recovered files are classified into two axes: the outcome of the reconstruction attempt and, when reconstruction is incomplete, the structural cause of the failure. The algorithm assigns an outcome and a failure cause at extent-resolution time.

2) Outcome categories

Complete recovery: All extent items associated with the file object identifier resolve to valid physical addresses, and the reconstructed content matches the original byte-for-byte. Inode metadata and directory paths are also recovered, along with their deletion status.

Overwritten-extent recovery: All extent items resolve to valid physical addresses, but one or more underlying data blocks have been reallocated and rewritten after deletion. Reconstruction produces a file of the correct logical length with inode metadata and path intact, but with content that does not match the original. This outcome is structurally indistinguishable from complete recovery without external verification.

Partial recovery: A strict subset of the file's extents resolves to valid physical addresses, while at least one extent cannot be mapped via the current chunk tree. Resolvable extents are written at their correct logical positions, and unresolvable extents are recorded in the audit log, and the corresponding regions are left as sparse holes, preserving the inode-declared total file size. Inode metadata and path are recovered.

Metadata-only recovery: Inode metadata and directory paths are recovered, but no extent items associated with the file resolve to valid physical addresses, and no content bytes are written. This outcome is structurally producible when the entire file falls outside the current chunk mapping, most commonly for single-extent files affected by chunk-tree turnover. It does not appear in the present dataset because no test file satisfied this exact condition.

Failed recovery: The file object identifier is observed in a historical filesystem tree, but neither inode metadata nor content can be reconstructed. Such a case is rare, as its main cause can be a discontinuity due to overwriting or reallocation of that block that occurs during the parsing of a B-Tree. Data of interest can also be missed from the enumeration target if the leaf or its parent nodes are not reachable due to the deletion of the subvolume, making its root unreachable using the current implementation.

D. FINAL IMAGE RECOVERY OUTCOMES

Recovery was performed on the final image (after all nine files were deleted). The recovered artifacts were validated using SHA-256 [29] comparison and metadata inspection.

The evaluation of the recovery algorithm was conducted on filesystem images where files were intentionally deleted from the user subvolume, and the recovery performance

TABLE II. Recovery Results From Final Image

File	Layout	Outcome	Failure Cause	SHA-256
binary.bin	Multi	Complete	—	Match
audio1.wav	Multi	Complete	—	Match
audio2.mp3	Multi	Complete	—	Match
image3.bmp	Single	Complete	—	Match
image5.jpg	Single	Complete	—	Match
file1.txt	Inline	Complete	—	Match
video1.mkv	Multi	Partial	Missing chunk mapping	Mismatch
video2.mp4	Multi	Partial	Missing chunk mapping	Mismatch
filex.txt	Single	Overwritten	Overwritten data block	Mismatch

reported in this work focuses on these controlled deletions. However, the algorithm also recovered additional deleted artifacts originating from other subvolumes, such as /var, including system-generated files deleted by normal operating system activity. These recoveries were not systematically documented across test images as they were incidental. As a result, they are not included in the quantitative evaluation presented in this paper. Nevertheless, this observation indicates that the proposed approach can identify and recover deleted artifacts across multiple Btrfs subvolumes.

Six files were recovered with complete cryptographic integrity. One file retained the correct metadata but produced a different hash, due to the overwriting of the original extent contents. Two multi-extent files were only partially reconstructable due to missing extent mappings.

Overall results:

- Full integrity recovery: 6/9
- Overwritten reconstruction: 1/9 (overwritten data block)
- Partial recovery: 2/9 (missing chunk mapping)
- Metadata-only recovery: 0/9 (category structurally producible)
- Failed recovery: 0/9

Given the small dataset, these counts are indicative of algorithm behavior under the tested conditions rather than statistically general recovery rates. Every non-complete outcome was attributable to a structural condition explicitly identified by the taxonomy above. The cause of the overwritten data block's failure is undetectable without external ground truth and is observable here only because the experimental setup preserves the original file hashes.

On the controlled evaluation dataset (9 files across 6 deletion cycles), full cryptographic integrity recovery was achieved for 6 files (66.7%). Given the small dataset size, these figures should be interpreted as indicative of algorithm behavior under the tested conditions rather than as statistically general recovery rates. Notably, all recovery failures were attributable to chunk-tree turnover or extent overwriting, which are structural conditions detectable at analysis time rather than to silent algorithm errors. No file produced an incorrect reconstruction without a corresponding detectable failure condition.

E. STRUCTURAL RECOVERABILITY

The file *video1.mkv* was recoverable in earlier intermediate images but not in the final image. Loss of recoverability did not occur immediately after deletion but only after later filesystem operations.

The chunk tree remained unchanged across the first five image acquisitions. After the sixth deletion stage, the chunk tree structure differed, and the logical-to-physical mapping for some extents was no longer present.

Repeated staged deletions were intentionally performed to trigger block group release and reallocation. This demonstrates that recovery failure in Btrfs is governed by chunk tree turnover rather than the deletion event itself. Metadata (inode data, extent locations, and path) persisted even when data blocks were replaced, or extents could not be mapped.

IX. EXPANDED EXPERIMENTAL EVALUATION

The previous experiment demonstrates the end-to-end behavior of the proposed method; it does not span the range of Btrfs storage forms, operational conditions, or filesystem activity profiles under which a forensic recovery tool must remain correct. This section extends the evaluation across multiple purpose-built filesystem images, each isolating a specific structural property, feature, or operational scenario relevant to deleted-file recovery.

Every case in this section was produced by a deterministic shell harness that constructs a raw Btrfs image, populates it according to the property under test, records a ground-truth manifest of every file (path, logical size, SHA-256 digest, observed extent count, and deletion flag) prior to the deletion event, performs the scripted deletions, and unmounts the image. The recovery algorithm is then executed on the resulting static raw image, and its output is compared against the manifest. A file is reported as recovered with full integrity only when the reconstructed content matches the recorded SHA-256 byte-for-byte, and the reconstructed path resolves to the original path recorded at construction time. Inode metadata recovery is verified by inspecting the timestamps and logical size written to the reconstructed file. All images were produced with `mkfs.btrfs` from `btrfs-progs` and processed by the recovery tool. In all cases, the recovery tool operated on the unmounted raw image and did not modify the input.

Across every case in this evaluation, full path reconstruction to the subvolume root and inode metadata recovery (creation, modification, change, and access timestamps, plus logical size) succeeded for every file classified as Complete, Overwritten-extent, or Partial. Only the failed outcome lacks both.

The remainder of this section presents each case in turn. For each image, we describe the isolated scenario, the construction procedure, the per-file recovery results, and a short analysis tying the outcomes back to specific algorithmic branches and to the failure-cause taxonomy.

A. CASE A: EXTENT LAYOUT COVERAGE

TABLE III. Case A File Categories and Constructed Sizes

Category	Count	Sizes	Construction method
inline	5	100, 500, 1024, 2048, 3584 B	Single sequential write below the inline threshold
single	5	64 KiB, 1 MiB, 16 MiB, 64 MiB, 128 MiB	Single sequential write per file
multi	3	128 MiB each	Random-order <code>pwrite+fsync</code> in 6 MiB chunks, ≈ 22 extents per file
sparse	3	1 GiB logical, ≈ 12 KiB physical each	<code>truncate -s 1G</code> then three 4 KiB writes at offsets 0, ≈ 500 MiB, ≈ 999 MiB
prealloc	3	64 MiB reserved, 16 MiB written	<code>fallocate -l 64M</code> then <code>dd</code> writing the first 16 MiB

1) Scenario

Btrfs represents file content in five structurally distinct forms: inline extents stored within the leaf node, single-extent files whose entire content resides in one data extent, multi-extent files whose content is spread across multiple data extents at arbitrary logical offsets, sparse files whose logical layout contains explicit holes, and prealloc files whose extent items reserve logical ranges that have never been written. The reconstruction routine contains a distinct code path for each of these forms, together with a final inode-size truncation step that materializes holes and unwritten ranges as zero-filled bytes. This case isolates layout handling in the reconstruction routine from any confounding effects of deletion cycles, filesystem aging, or chunk-tree turnover. A single image is constructed, populated with files spanning every layout, subjected to a single deletion event, and recovered.

2) Image Construction

A 2 GiB raw image was formatted with the default `mkfs.btrfs` parameters and mounted with the option `max_inline=4096` to permit inline extents of up to 4 KiB. A single-user subvolume `test_subvol` was created, and nineteen files were written across five category directories as summarized in Table III.

After population and `btrfs filesystem sync`, eleven files were removed in a single deletion event covering approximately half of each category: three of five inline files, three of five single-extent files, two of three multi-extent files, two of three sparse files, and one of three prealloc files. The filesystem was then synchronized and unmounted, producing the static image submitted to the recovery tool.

3) Recovery Results

All eleven deleted files were reconstructed to their declared logical size with byte-exact content as seen in Table IV. Path reconstruction resolved each file to the subvolume root through the recorded parent identifiers, and inode metadata, including creation, modification, change, and access timestamps, was written to each reconstructed file. No file required the unresolved-extent or PREALLOC-skip branches of Algorithm 8 to be invoked in a way that left unwritten

regions, since every extent referenced by every deleted inode was resolved through the current chunk tree.

Case A exercises each of the conditional branches of Algorithm 8 with at least one deleted file. The three recovered inline files traverse the inline-extent branch, which reads content directly from the host leaf without consulting the chunk tree. The three single-extent files validate the standard extent-read branch across four orders of magnitude in size. The two multi-extent files validate the offset-sorted reassembly logic: each file's 22 extents were reassembled in the correct logical sequence solely from each extent item's `file_offset` key. The two sparse files validate the hole-materialization path, in which the per-extent loop seeks to each extent's logical offset, writes the small data region, and leaves the intervening logical range untouched. The final truncate to the inode-declared size zero-fills the holes, producing a 1 GiB output whose SHA-256 matches the original. The recovered prealloc file validates the PREALLOC-skip branch combined with the final truncation.

B. CASE B: COMPRESSION COVERAGE

1) Scenario

Btrfs supports transparent compression on a per-extent basis. When a filesystem is mounted with a `compress` option, the kernel applies a heuristic to each write and stores the result either as a compressed extent (with an algorithm identifier and a decompressed-length field in the extent item) or, when the data is judged incompressible, as a plain extent. Algorithm 8 handles compressed extents by reading the compression-type field from the extent data item and decompressing the on-disk bytes before placing them at the extent's file offset. This case isolates that branch and additionally verifies that the algorithm continues to behave correctly when the filesystem heuristic stores a file uncompressed within a `compress=zstd` mount.

2) Image Construction

A 2 GiB raw image was formatted with the default `mkfs.btrfs` parameters and mounted with `compress=zstd,max_inline=4096`. Eleven files were written across four category directories, as summarized in Table V. Compressible files were generated from a repeating ASCII pattern that Zstd reduces by approximately fifty times. Incompressible files were generated from `/dev/urandom` so that the kernel heuristic would skip compression at write time. After population and synchronization, seven files were removed in a single deletion event: two of three compressed inline files, two of three compressed single-write files, two of three compressed multi-write files, and one of two heuristic-uncompressed files. The filesystem was then synchronized and unmounted.

3) Recovery Results

All 7 deleted files recovered, with SHA-256 matches as seen in Table VI. The compressed inline, compressed multi-

extent, and heuristic-uncompressed branches of Algorithm 8 are each exercised by at least one file.

The two `single_zstd` files show that Btrfs splits compressed extents at a fixed 128 KiB decompressed boundary: the 1 MiB file is stored as eight extents, and the 4 MiB file as thirty-two, despite being written as a single sequential write. Reconstruction handles this because the offset-sorted loop is agnostic to the source of fragmentation. The `uncompr_random` file confirms that the compression-type field is read per extent rather than inferred from the mount state.

C. CASE C: SUBVOLUME AND SNAPSHOT COVERAGE

1) Scenario

This case probes per-subvolume classification scope, shared-extent persistence across snapshots, and whole-subvolume deletion. The algorithm classifies deleted files independently within each subvolume, and reconstruction must succeed when a deleted file's extents remain alive through a reflink in another subvolume.

2) Image Construction

Four subvolumes were created as summarized in Table VII. After population, three file-level deletions and one whole-subvolume deletion were performed. The snapshot is taken immediately after `src` is populated, so `src_snap/file_X` initially shares its extent with `src/file_X` for X in A, B, C.

3) Recovery Results

The three file-level deletions across three distinct subvolumes all recovered, complete with SHA-256 match, validating per-subvolume scoping as seen in Table VII. The `src_snap/file_C` recovery is the explicit shared-extent case. The file was removed from the snapshot while the source copy in `src` remained alive. The underlying extent persisted because `src` still referenced it, and reconstruction was resolved through the live mapping.

The whole-subvolume deletion resulted in two failures. After `btrfs subvolume delete will_delete` is committed, the corresponding `root_item` is removed from the active root tree, and the current implementation enumerates subvolumes exclusively from that tree. The subvolume is therefore not discoverable as a recovery scope, and `file_G` and `file_H` are not reached. This is the structural failure mode where we describe the proposed extension: parse `root_item` and `root_ref` entries from each historical root tree discovered in Algorithm 3 to construct a per-generation subvolume index, enabling traversal of subvolumes that no longer exist in the active root tree. The extension is well-defined and is left for future work.

For comparison, `btrfs restore` recovers files from a deleted subvolume of this kind in practice. It does so by traversing all reachable tree blocks without performing subvolume enumeration or deletion classification, copying any file content it can read. The forensic tradeoff is that the

TABLE IV. Case A Per-File Recovery Results (Deleted Files Only)

File	Layout	Orig size (B)	Rec size (B)	Extents (exp.)	Extents (rec.)	SHA-256	Outcome
inline_100.bin	inline	100	100	1	1	Match	Complete
inline_1024.bin	inline	1,024	1,024	1	1	Match	Complete
inline_3584.bin	inline	3,584	3,584	1	1	Match	Complete
single_65536.bin	single	65,536	65,536	1	1	Match	Complete
single_16777216.bin	single	16,777,216	16,777,216	1	1	Match	Complete
single_134217728.bin	single	134,217,728	134,217,728	1	1	Match	Complete
multi_a.bin	multi	134,217,728	134,217,728	22	22	Match	Complete
multi_c.bin	multi	134,217,728	134,217,728	22	22	Match	Complete
sparse_1.bin	sparse	1,073,741,824	1,073,741,824	3	3	Match	Complete
sparse_3.bin	sparse	1,073,741,824	1,073,741,824	3	3	Match	Complete
prealloc_2.bin	prealloc	67,108,864	67,108,864	1	1	Match	Complete

TABLE V. Case B File Categories and Constructed Sizes

Category	Count	Sizes	Construction method
inline_zstd	3	500, 2048, 3500 B	Compressible ASCII pattern, single sequential write below inline threshold
single_zstd	3	128 KiB, 1 MiB, 4 MiB	Compressible ASCII pattern, single sequential write
multi_zstd	3	768 KiB each	Compressible ASCII pattern, random-order pwrite+fsync in 128 KiB chunks (6 extents/file)
uncompr_random	2	2 MiB, 8 MiB	/dev/urandom; written within a compress=zstd mount, heuristic stores uncompressed

deleted-vs-live classification, full path attribution, and per-subvolume scoping that motivate the present method are lost in exchange. Once the proposed extension in Section X.E is implemented, the same recovery becomes achievable under the deterministic-classification model described in this paper.

D. CASE D: CLASSIFICATION SEMANTICS AND METADATA EDGE CASES

1) Scenario

This case exercises the classification and path-reconstruction routine against operations such as rename and intra-subvolume move, hard links under full and partial deletion, truncation prior to deletion, and recursive deletion of parent directories at shallow and deep nesting levels. Each operation alters the relationship between an object identifier, its directory entries, and its extent items in a structurally distinct way.

2) Image Construction

A 1 GiB raw image was formatted with default `mkfs.btrfs` parameters and mounted with `max_inline=4096`. A single subvolume `test_subvol` was created and populated as summarized in Table VIII. File contents were drawn from `/dev/urandom` so that recovery correctness is decided strictly by SHA-256 comparison.

The harness proceeds in phased operations, each followed by `btrfs filesystem sync` so that successive states land in distinct generations and historical-tree behavior under recovery becomes observable. After initial creation, files in the rename and move categories are renamed or moved to

their final paths. The two truncate-category files are then truncated, one to zero bytes and one to 262,144 bytes (from an original 1,MiB). A ground-truth manifest is recorded at this point, capturing each file's final path, logical size, and SHA-256. Deletions are then performed: direct `rm` for the rename, move, and truncate targets; removal of both names for the full-hardlink case and only one name for the partial-hardlink case; and `rm -r` on the `deleted_dir` and deep hierarchies. The filesystem is synchronized and unmounted, producing the static image submitted to the recovery tool.

3) Recovery Results

Per-file results for the nine deleted entries are reported in Table IX. Seven are reconstructed with byte-exact content and correct path, and two were not recovered.

The rename and move cases both recover under their post-operation paths. The historical directory item bearing the pre-rename or pre-move name is observed during traversal of older generations, but its object identifier matches the entry seen at the deletion-time generation, so the prior name is implicitly discarded, and the final path is the one emitted.

Both `link_a.bin` and `link_b.bin` reference the same object identifier, and the directory-item recording step in Algorithm 7 is gated on whether the object identifier has already been added to the deleted-candidate set. Only the first directory item encountered for that identifier is retained, so exactly one of the two paths is reconstructed. In this run, `link_b.bin` was the first encountered and was recovered; `link_a.bin` was dropped. Content recovery is unaffected because both names refer to the same extent.

The partial-hardlink case confirms the predicate's correct non-classification behavior. `link_y.bin` is removed, but the underlying inode persists through `link_x.bin`, so the object identifier remains in the present-file set, and the predicate does not classify it as deleted. The removed name is therefore not emitted as a recovery target. Recovering the name of the removed link would require analysis of historical inode-reference entries and is out of scope.

Both truncated variants recover at their post-truncate logical sizes (0,B and 256,KiB respectively). The inode item observed at deletion time records the truncated size, and Algorithm 8 bounds reconstruction by that size. The pre-truncate extents are not re-attached: extents beyond the

TABLE VI. Case B Per-File Recovery Results (Deleted Files Only)

File	Layout	Orig size (B)	Rec size (B)	Extents (exp.)	Extents (rec.)	SHA-256	Outcome
inline_zstd_500.bin	inline (zstd)	500	500	1	1	Match	Complete
inline_zstd_3500.bin	inline (zstd)	3,500	3,500	1	1	Match	Complete
single_zstd_1048576.bin	single (zstd)	1,048,576	1,048,576	8	8	Match	Complete
single_zstd_4194304.bin	single (zstd)	4,194,304	4,194,304	32	32	Match	Complete
multi_zstd_a.bin	multi (zstd)	786,432	786,432	6	6	Match	Complete
multi_zstd_c.bin	multi (zstd)	786,432	786,432	6	6	Match	Complete
uncompr_random_8388608.bin	single (none)	8,388,608	8,388,608	1	1	Match	Complete

TABLE VII. Case C Per-File Recovery Results (Deleted Files Only)

File	Subvolume	Orig size (B)	Rec size (B)	Extents (exp.)	Extents (rec.)	SHA-256	Outcome	Failure cause
file_B.bin	src	65,536	65,536	1	1	Match	Complete	—
file_C.bin	src_snap	65,536	65,536	1	1	Match	Complete	—
file_F.bin	other	65,536	65,536	1	1	Match	Complete	—
file_G.bin	will_delete	65,536	—	1	0	Mismatch	Failed	Subvolume not enumerable
file_H.bin	will_delete	65,536	—	1	0	Mismatch	Failed	Subvolume not enumerable

TABLE VIII. Case D File Categories and Construction

Category	Count	Size	Construction / operation
rename	2	1, KiB	Created, then renamed in place; one later deleted, one kept
move	1	1, KiB	Created in <code>dir_a</code> , moved to <code>dir_b</code> , then deleted
hl_full	2	64, KiB	Two hard links to one inode; both names removed
hl_partial	2	64, KiB	Two hard links to one inode; one name removed, the other retained
truncate	2	1, MiB	Truncated (one to 0, B, one to 256, KiB) before deletion
deleted_dir	1	64, KiB	File inside a directory removed via <code>rm -r</code>
deep	1	64, KiB	File at depth 4 inside a directory tree removed via <code>rm -r</code>

inode-declared length are not selected, which is the intended behavior of inode-bounded reconstruction.

The deleted-directory and deeply nested cases both recover their leaf files to complete paths. The parent directories themselves are not reconstructed as objects, but their directory items remain present in historical leaves and are consulted by the parent-lookup recursion, allowing path reconstruction to traverse the deleted directories transparently. This holds at both shallow (depth 1) and deep (depth 4) nesting.

E. CASE E: HEAVY CHURN AND COMPARATIVE TRAVERSAL COST

1) Scenario

This case stresses the two axes on which the proposed traversal is designed to scale: the number of committed root-tree generations, and the number of CoW-shared internal and leaf nodes that any deletion-aware traversal must visit. Algorithm 2 maintains a per-traversal cache of visited node logical addresses (*VisitedNodes*), and Algorithm 3 retains only the highest-level block per observed generation, so that the cost of walking historical filesystem state across all generations

is bounded by the number of distinct nodes rather than by the number of generations. The image is constructed so that this scaling property is observable, and so that any recall difference between recovery tools is attributable to traversal strategy rather than to data destruction.

To eliminate data destruction as a confounding variable, the image is configured to preserve deleted data on disk. Files are written with `max_inline=0` so that all data resides in data block groups rather than in metadata leaves, the image is sparsely provisioned at 16 GiB to suppress allocator pressure, deletions are scattered across many directories so that no block group is emptied, and no balance, discard, or `fstrim` operation is performed at any point. Under these conditions, the chunk tree remains stable across the entire run.

2) Image Construction

A 16 GiB raw image was formatted with `mkfs.btrfs -n 16384` and mounted with `noatime,max_inline=0` and `commit=999`. The `commit=999` option suppresses periodic auto-commits so that the committed-generation count is determined by explicit `btrfs filesystem sync` calls in the harness rather than by background activity. Two subvolumes were created: a target subvolume `churn_subvol` populated with the files under test, and a scratch subvolume containing a single heartbeat file used to inflate the committed-generation count without touching any target data.

Forty directories were created in the target subvolume, each populated with twenty-five files drawn from `/dev/urandom`, for a total of 1000 files. File sizes were drawn from three tiers: small single-extent (8, KiB–64, KiB), medium single-extent (128, KiB–1, MiB), and large multi-extent (4–8 chunks of 256, KiB–1, MiB each, with `fsync` between chunks to force separate extents). One filesystem commit was issued per directory during creation, producing forty distinct creation generations. A read-only snapshot of the populated subvolume was then taken.

TABLE IX. Case D Per-File Recovery Results (Deleted Files Only)

File	Category	Orig size (B)	Rec size (B)	Extents (exp.)	Extents (rec.)	SHA-256	Outcome / Cause
rename/rename_then_delete.bin	rename	1,024	1,024	1	1	Match	Complete
move/dir_b/moved_then_delete.bin	move	1,024	1,024	1	1	Match	Complete
hl_full/link_b.bin	hl_full	65,536	65,536	1	1	Match	Complete
hl_full/link_a.bin	hl_full	65,536	—	1	0	—	Not emitted (single dir-item retained per object ID)
hl_partial/link_y.bin	hl_partial	65,536	—	1	0	—	Not classified as deleted (inode persists via surviving link)
truncate/truncate_to_zero.bin	truncate	0	0	0	0	Match	Complete
truncate/truncate_partial.bin	truncate	262,144	262,144	1	1	Match	Complete
deleted_dir/inside.bin	deleted_dir	65,536	65,536	1	1	Match	Complete
deep/a/b/c/leaf.bin	deep	65,536	65,536	1	1	Match	Complete

TABLE X. Case E File Population and Deletion Distribution

Category	Total	Deleted	Surviving
Single-extent, <,64,KiB	642	196	446
Single-extent, 64,KiB–1,MiB	267	84	183
Multi-extent, ≥,1,MiB	91	20	71
Total	1000	300	700

TABLE XI. Case E Aggregate Recovery Outcome (Deleted Files Only)

Layout	Deleted	Recovered	SHA match
Single-extent, <,64,KiB	196	196	196
Single-extent, 64,KiB–1,MiB	84	84	84
Multi-extent, ≥,1,MiB	20	20	20
Total	300	300	300

Thirty deletion cycles followed. Each cycle removed ten files chosen at random from a pre-shuffled deletion pool, followed by twelve heartbeat rewrites of the scratch file, each separated by a filesystem commit. With these parameters, the design lower bound on the committed root-tree generations is $40 + 1 + 30 \times (1 + 12) = 431$. Deletions and surviving files by category are summarized in Table X.

3) Recovery Results

All 300 deleted files were reconstructed, with timing, memory, and similar cost analysis in Table XII, with byte-exact content (SHA-256 match against the pre-deletion manifest) and complete path resolution to the subvolume root, across all thirty deletion cycles. No file required the unresolved-extent, PREALLOC-skip, or ownership-discontinuity branches of Algorithm 8. The aggregated outcome is reported in Table XI.

4) Comparative Traversal Cost

The proposed analyzer is implemented in Python and the reference pipeline (`btrfs_find_root` and `btrfs_restore`) is compiled C from `btrfs-progs`. The timing comparison therefore understates the traversal-cost asymmetry, since the Python implementation incurs interpreter overhead absent from the C baseline.

To isolate the cost of historical-state traversal from the cost of file extraction, the proposed analyzer was compared

TABLE XII. Case E Comparative Cost (Deleted-File Enumeration Only)

Metric	<code>find-root + restore</code>	Our implementation
Wall-clock time (s)	4.28	0.94
Peak RSS (KiB)	136,968	310,508
Disk read (MiB, <code>ru_inblock</code>)	466.6	267.1
Node-level traversal cache	no	yes
Classifies deleted vs. present	no	yes

against a pipeline-based on the existing `btrfs-progs` utilities, configured to perform deleted-file enumeration only. The reference pipeline invokes `btrfs-find-root -a` to discover candidate historical root tree blocks, then iterates `btrfs restore -t -Divv` (with `-D` forcing dry-run, so no content is written) over each discovered root, collecting the union of reported file paths. For parity, the analyzer was also run with content-write disabled; its output therefore consists of the set of deleted file paths together with the in-memory inode and extent metadata, it collects during traversal, but no extracted file data is written to disk.

Both tools were executed cold (page cache dropped before each invocation), and figures are reported as the median of ten runs. The analyzer completed in a median of 0.94 s with a peak resident set size of 310,508 KiB, while the reference pipeline completed in 4.28 s with a peak resident set size of 136,968 KiB (Table XII). The wall-time difference is consistent with the traversal-cost asymmetry between the two approaches: the reference pipeline invokes `btrfs restore` once per discovered root with no cross-invocation node cache, so each CoW-shared internal or leaf node is re-walked once per generation visited, whereas the analyzer's *VisitedNodes* cache (Algorithm 2) visits each shared node at most once across the entire historical-tree traversal. The higher resident set size for the analyzer reflects the in-memory *InodeItemSet* and *ExtentItemSet* collected during traversal (Algorithm 5); the reference pipeline retains only the current file-path list. This is a deliberate trade-off in the proposed design: deletion classification and extent-accurate reconstruction (Algorithm 8) both require global metadata visibility, which has a memory cost that pure path enumeration does not incur.

All timings were obtained with both tools reading the

same on-disk image under a cold page cache, with the kernel cache dropped before each run, and medians are reported across 10 runs. That the cache was genuinely cold on every run is confirmed by the per-run file-system input counts (`ru_inblock`), which remained at hundreds of MiB rather than collapsing to zero. These same counts provide a cache-independent measure of traversal redundancy where the reference pipeline read 466.6 MiB from the device versus 267.1 MiB for the analyzer, which is approximately 43% fewer blocks because it invokes `btrfs restore` once per discovered root with no cross-invocation node cache, re-walking each CoW-shared node once per generation, whereas our implementation's VisitedNodes cache (Algorithm 2) visits each shared node at most once across the entire historical-tree traversal. The wall-time ratio and the disk-read ratio are consistent, and both follow directly from this asymmetry.

A within-tool A/B comparison (cache enabled vs. disabled in the same binary) is functionally equivalent to the comparison already reported. With the VisitedNodes cache disabled, our driver would re-walk each discovered root independently, which is precisely the per-root invocation pattern of `btrfs_find_root + btrfs restore`: each generation processed without shared traversal state, and each CoW-shared node re-read once per generation visited. The metadata-collection and reconstruction paths downstream of traversal are not exercised by the enumeration-only configuration, so the measured gap is attributable to traversal redundancy alone. The `ru_inblock` counts confirm this directly: the 466.6 MiB vs. 267.1 MiB difference reflects repeated reads of CoW-shared nodes under per-root invocation, which is the mechanism the VisitedNodes cache eliminates.

Functional differences between the two outputs are the absence of deleted/present classification in the reference pipeline, loss of timestamps and ownership in `btrfs restore` dry-run output, its handling of read-only snapshots, and the complexity of traversal of the filesystem tree.

F. CASE F: COMPARATIVE EVALUATION AGAINST EXISTING TOOLS

1) Scenario and Rationale

The preceding cases evaluate the proposed method in isolation using pseudorandom file content, which is sufficient when correctness is decided by SHA-256 identity, but places signature-based carvers at an artificial disadvantage, since random payloads expose no recoverable file headers. To compare the proposed method against existing tools on equal terms, a dedicated image was constructed from valid, well-formed files (PDF, JPEG, PNG, ZIP, MP4, and text) carrying the correct magic bytes, written across the same extent layouts exercised in Cases A and B: inline, single-extent, multi-extent (fragmented), sparse, prealloc, and deeply nested paths. A subset spanning every layout was then deleted in a single event, and the image was synchronized and unmounted. This construction gives carvers a fair opportunity to reconstruct contiguous content while still testing path, metadata, and deletion-classification recovery.

2) Tools and Invocation

Seven configurations were compared, all operating on the same unmounted raw image: (i) `foremost` and (ii) `PhotoRec`, signature-based carvers that reconstruct files from content and do not consult filesystem metadata; (iii) `btrfs-cue`, which samples the image to identify the filesystem, scans it to rebuild a metadata store heuristically, and exposes a FUSE view from which, per its documentation, only files no larger than one filesystem block can be copied; (iv) `btrForensics` and (v) the FKIE-CAD Sleuth Kit fork, both Sleuth-Kit-based parsers whose published capability is forensic analysis of the current filesystem and named subvolumes/snapshots, with the latter additionally providing multi-device pool analysis; (vi) a native pipeline combining `btrfs-find-root` with `btrfs restore -t` and (vii) the proposed method.

We note that tools (iv) and (v) are not designed for deleted-file recovery. They are included to characterize the gap between live-filesystem parsing and deletion-aware reconstruction, not to assess them outside their intended scope. For the FKIE-CAD fork, the pool-level listing and extraction routines (`BTRFS_POOL::fls`, `istat`, and `icat`) declare a generation argument, but the argument is not referenced in the bodies of these routines: each operates on the active filesystem tree, optionally re-pointed to a subvolume or snapshot selected by name, and the root filesystem identifier is fixed at initialization. The implementation, therefore, surfaces the current filesystem state and live subvolumes/snapshots, and does not perform historical-root selection or deleted-versus-present classification.

3) Evaluation Axes

Each tool is scored on the axes the proposed method targets: byte-exact content (SHA-256 against the pre-deletion manifest), original file path, inode metadata (timestamps and logical size), and explicit deleted-versus-present classification.

4) Analysis

The two tables (Table XIII and XIV) separate content recall from forensic completeness. Carvers recover contiguous, signature-bearing files with correct content, but produce no path, metadata, or deletion status. They degrade to partial output on fragmented multi-extent files, and recover nothing for inline files, which reside within metadata leaves rather than in the data block groups [15], and for sparse and prealloc files, which expose no contiguous signature. The Sleuth-Kit-based tools parse the live filesystem faithfully, but do not reach the deleted instances under test, consistent with their design scope. The native `btrfs restore` pipeline, directed at a historical root, reconstructs content and paths for the reachable files, but copies every reachable object indiscriminately, applies no deleted-versus-present classification, and omits inode metadata under default invocation, so its output is content recovery rather than forensic reconstruction. `btrfs-cue` rebuilds a heuristic metadata view, but by its documented design, extracts only files within a single block.

TABLE XIII. Capability Comparison for Deleted-File Recovery

Capability	FM	PR	BC	BF	FK	BR	Ours
Operates on unmounted raw image	✓	✓	✓	✓	✓	✓	✓
Recovers deleted-file content	P	P	P	–	–	✓	✓
Byte-exact (hash-verifiable) content	P	P	–	–	–	✓	✓
Recovers file name / path	–	–	P	–	–	✓	✓
Recovers inode metadata / timestamps	–	–	P	–	–	P	✓
Deleted-vs-present classification	–	–	–	–	–	–	✓
Historical (old-root) traversal	–	–	P	–	–	✓	✓
Multi-extent (fragmented) reassembly	–	P	–	–	–	✓	✓
Inline-extent recovery	–	–	P	–	–	✓	✓
Sparse / prealloc layout fidelity	–	–	–	–	–	✓	✓
Multi-device / pool input	–	–	–	P	✓	P	P

FM: foremost; PR: PhotoRec; BC: btrfs; BF: btrForensics; FK: FKIE-CAD TSK fork; BR: btrfs-find-root+btrfs restore. ✓ supported; P partial or conditional; – not supported. Columns BF and FK reflect deleted-file recovery only; both parse live filesystem state effectively, which this axis set does not credit. Orphan-item-based approaches [15] are not included as a separate column in Table XIII because they were characterized analytically rather than released as a recovery tool.

TABLE XIV. Per-File Recovery Outcome on the Comparison Image

File	Layout	FM	PR	BC	BF	FK	BR	Ours
inline_1.jpg	Inline	C	P	–	–	–	C	✓
inline_2.pdf	Inline	C	–	–	–	–	C	✓
inline_3.txt	Inline	–	P	–	–	–	C	✓
single_1.png	Single	C	C	–	–	–	C	✓
single_2.zip	Single	C	C	–	–	–	C	✓
single_3.mp4	Single	–	C	–	–	–	C	✓
frag_1.jpg	Multi	–	P	–	–	–	C	✓
frag_2.mp4	Multi	–	P	–	–	–	C	✓
sparse_1.bin	Sparse	–	–	–	–	–	C	✓
sparse_3.bin	Sparse	–	–	–	–	–	C	✓
prealloc_2.bin	Prealloc	–	–	–	–	–	C	✓
nested_1.jpg	Nested	C	C	–	–	–	C	✓

Tool abbreviations as in Table XIII. ✓ full forensic recovery (byte-exact content, path, inode metadata, and deletion status); C content recovered but lacking path, metadata, or deletion classification; P partial or corrupted content; – not recovered or deleted instance not surfaced.

The proposed method recovers every deleted file across all layouts with byte-exact content, full path resolution, inode metadata, and explicit deletion status, which is the combination of the comparison, is intended to isolate.

X. LIMITATIONS AND FUTURE SCOPE

The proposed method is designed to recover deleted files from valid, structurally consistent Btrfs filesystem images. It operates mainly by parsing logical filesystem metadata and does not attempt filesystem repair or recovery from corrupted metadata structures.

A. OVERWRITTEN METADATA AND OWNERSHIP DISCONTINUITIES

During traversal of historical filesystem trees, it may be observed that certain child nodes referenced from historical roots do not meet the generation, block pointer, or level condition with respect to their parent. In such cases, it indicates that the node has been overwritten and repurposed by a later filesystem state. When this occurs, the logical linkage between the historical filesystem tree and the overwritten node is irreversibly lost. In such cases, the algorithm terminates the traversal of that branch.

Because the proposed method relies on consistent ownership and preserved structural relationships via CoW semantics, filesystem metadata overwritten by subsequent writes

cannot be recovered using this approach. This limitation is inherent to the filesystem design and not specific to the implementation.

B. DEEP LEAF SCAN AS A PARTIAL RECOVERY EXTENSION

One possible extension to address ownership discontinuities is a deep leaf scan of the filesystem image. In this approach, leaf nodes that were not reached through filesystem tree traversal could be examined directly for directory items that have not been previously observed. While such a scan may enable partial recovery of deleted files when tree linkage has been lost, it cannot reliably reconstruct directory paths or confirm the completeness of recovery.

Because the present work focuses on complete and structurally accurate file recovery through filesystem tree parsing, deep leaf scanning is not implemented and is considered future work.

C. OUTPUT AND STORAGE CONSIDERATIONS

The current implementation reconstructs and writes all recoverable file contents to local storage. This behavior reflects an implementation choice made to validate the correctness of the recovery algorithm rather than a requirement of the method itself. In practical forensic workflows, recovery may

be selectively performed using metadata mappings produced by the algorithm without reconstructing all file contents.

D. CHUNK-TREE TURNOVER

In Btrfs, every logical address recorded in metadata must be converted to a physical address through the chunk tree, whose root pointer is stored in the superblock and updated on every transaction commit. Unlike filesystem trees, whose historical generations remain discoverable by parsing their respective historical root trees, the chunk tree is referenced exclusively from the superblock, like the root tree. The chunk tree nodes themselves are written using the same CoW mechanism as other tree nodes and may remain physically present on disk, but since they are directly referenced in the superblock, their old versions must be scanned for with their respective owner ID, as done with the root tree.

The chunk root is not affected under moderate filesystem activity. Deleted files remain reachable through historical metadata nodes whose extent references continue to resolve via the current chunk mapping. However, under conditions that include automatic reclaim of empty block groups, balance operations, device addition or removal, and bulk deletion sequences that empty a block group and place the allocator under pressure to reuse the released logical range, these assumptions fail. When the allocator releases a block group and a new chunk subsequently occupies that logical range, the current chunk tree no longer carries a mapping entry for the original logical region, even though the physical bytes referenced by historical metadata may remain intact. The result is not direct data destruction but loss of the ability to map all logical addresses to physical addresses referenced by that block.

This failure mode is detected during recovery when an extent reference produced from a historical filesystem tree resolves to a logical address that has no entry in the bootstrapped chunk tree. Chunk-tree turnover prevents reconstruction from proceeding for the affected extents, since no physical location can be derived for the logical reference, as observed for `video1.mkv` and `video2.mp4` in Table II.

The chunk tree remained unchanged across the first five image acquisitions and changed only after the sixth deletion cycle. The deletions performed in the final cycle removed the last extents in a previously occupied block group, triggering the reclaim of that block group and the subsequent reallocation of its logical range. The historical metadata nodes referencing the released extents remained on disk and were correctly enumerated by the traversal procedure, but the logical addresses they carried could no longer be translated through the current chunk tree.

Recovery from chunk-tree turnover would, in principle, require reconstructing a historical chunk tree state corresponding to the generation at which the affected extents were alive. Historical chunk tree nodes are themselves preserved by CoW and can be located on disk by scanning for blocks whose header ownership field identifies them as belonging to the chunk tree, analogous to the historical root

tree discovery procedure formalized in Algorithm 3. The `btrfs rescue chunk-recover` utility distributed with `btrfs-progs` performs an analogous scan as part of its repair workflow and demonstrates the feasibility of node-level discovery, but lacks the ability to integrate it into the rest of the recovery process. The substantive difficulty in adapting this approach for forensic recovery is not discovery but selection. A recovered chunk tree state must correspond temporally to the historical filesystem tree being processed, which requires correlating generation numbers across two CoW update streams that progress independently. In the absence of an explicit cross-tree generation index, the selection of an incorrect chunk tree state may produce logically plausible but semantically incorrect logical-to-physical translations, which would undermine the deterministic recovery property that motivates the present method. For this reason, automated reconstruction of the historical chunk tree state is left as future work. An extension that combines scan-based historical chunk node discovery, modeled on the approach used by `chunk-recover`, with an explicit generation-correspondence and consistency-validation stage would constitute a well-defined direction for addressing this limitation.

E. DELETED SUBVOLUME RECOVERY

When a subvolume is deleted via `btrfs subvolume delete` and the deletion is committed, the corresponding `root_item` is removed from the active root tree in the same transaction. Inspection of a post-deletion filesystem image with `btrfs inspect-internal dump-tree` confirms that neither the subvolume identifier, nor the pointer to its filesystem tree root, nor any `root_ref` item naming it remains present in the active root tree.

The present implementation enumerates subvolumes exclusively from the active root tree, and is therefore unable to discover, name, or traverse a fully-deleted subvolume.

Recovery is feasible in principle but requires an extension to the subvolume enumeration stage with per-generation subvolume indexing during historical root tree traversal. Algorithm 3 already discovers historical root tree blocks indexed by generation. The extension parses each historical root tree's `root_item` and `root_ref` entries and constructs a per-generation index of subvolumes that existed at that generation, recording their tree-root logical addresses and reference names.

F. SCALABILITY

The method scales along three axes relevant to forensic deployment: raw image size, filesystem history (number of committed generations and deletion churn), and peak memory. We address each in turn, since the caching strategy of Algorithm 2 governs the first two and trades against the third.

a: Image size.

Analysis cost is governed by the volume of occupied metadata, not by raw device capacity. Historical-root discovery (Algorithm 3) scans only the regions the chunk tree maps

to Btrfs tree blocks, and traversal cost is proportional to the number of distinct nodes reached, not to the number of data bytes on disk. A sparsely populated multi-terabyte image and a densely populated one of the same metadata footprint therefore incur comparable analysis costs. The data extent bytes themselves are read only during reconstruction, lazily and once per resolved extent, so image size influences I/O only to the extent that recovered file content must be read back.

b: History and churn: cached vs. uncached traversal.

The dominant scalability concern in an aged or heavily churned filesystem is the growth in committed generations: every deletion forces a copy-on-write ripple from the affected leaf to the root, producing a new historical root while leaving the bulk of the tree physically shared. A naive approach that re-traverses each discovered root independently re-walks every shared node once per generation, giving

$$T_{\text{naive}} = \Theta(Gn) \quad (7)$$

for G generations over a tree of n leaves. The VisitedNodes cache (Algorithm 2) admits each shared logical node at most once across the entire historical tree traversal, reducing the cost to the size of the union of all generations,

$$T_{\text{cache}} = O(n + G\bar{m} \log_f n) \quad (8)$$

where $\bar{m}$ is the mean number of leaves modified per commit. The speedup

$$S = \frac{GN}{N + (G-1)\bar{\delta}} \xrightarrow{G \rightarrow \infty} \frac{N}{\bar{\delta}} \quad \bar{\delta} \leq \bar{m}(h+1) \quad (9)$$

is most pronounced precisely under the conditions that defeat the naive approach: the more history and churn an image carries, the larger the fraction of redundant work the cache eliminates. When per-commit churn satisfies $\bar{m} \log_f n \ll n$ —the common case, in which a handful of files change per commit, while the tree remains large—cached traversal cost is effectively $\Theta(n)$, independent of generation count.

c: Memory.

The cost of this traversal strategy is paid in memory. Because deletion classification and extent-accurate reconstruction both require global visibility of metadata (directory, inode, and extent items for one object may reside in different leaves and generations), Algorithm 5 collects the inode and extent item sets across all traversed generations before filtering. Peak resident memory therefore scales as $O(F + E)$ in the number of files F and extents E in the union of the current and historical trees, rather than with image size. This is visible in Case E as a higher resident set than the path-enumeration-only reference pipeline. For filesystems with very large object counts, this growth is the binding scalability constraint. A streaming or on-disk-indexed variant of the metadata collection stage is a well-defined direction for removing the in-memory bound entirely, and is left as future work.

G. BEHAVIOR UNDER DEGRADED INPUT

The method assumes a valid, internally consistent filesystem image and does not attempt to repair corrupted or partially damaged filesystems. This subsection characterizes how the algorithm behaves when that assumption is violated, distinguishing the cases in which it terminates, skips an affected region, or degrades into a reported failure outcome. The analysis concerns the soundness of the tool's response to damage rather than recovery from it: the governing property, established below, is that corruption reduces the number of recovered files and surfaces explicit failure labels, but never causes the algorithm to emit a confident yet incorrect reconstruction.

Damage is considered for each on-disk structure that the method depends on. When the primary superblock is unreadable or fails its consistency checks, the algorithm terminates before any recovery is attempted. Btrfs maintains backup superblock copies at fixed offsets; a fallback to a surviving copy is a straightforward and forensically conventional extension that would allow analysis to proceed when only the primary copy is damaged. When the chunk tree is partially damaged, extents whose logical addresses cannot be translated are routed to the unresolved-extent branch of Algorithm 8, and the affected files are reported as Partial or Metadata-only, with the unmapped addresses recorded in the audit log. Loss of the entire chunk mapping prevents translation altogether, and the algorithm terminates.

Corruption of the active root tree is partially tolerated by construction. Historical root trees are discovered by scanning the image for blocks whose ownership field indicates they are root-tree nodes, a procedure that does not depend on the active root pointer stored in the superblock. Consequently, damage confined to the current root tree does not necessarily prevent recovery, since historical roots and the filesystem trees they reference may remain discoverable and traversable. Within an individual tree, a node or leaf whose header cannot be parsed, or whose generation, level, or pointer validation conditions are not met, causes traversal of that branch to terminate. The items in the affected node are not enumerated, but no data from it is fabricated or misattributed. Damage to individual metadata items maps onto the same taxonomy, where a malformed inode item yields a Failed outcome for the corresponding object identifier, while an extent item carrying an invalid logical address is treated as unmappable and contributes to a Partial or Metadata-only outcome. Regions of the image that are unreadable or zeroed, whether due to bad sectors or a truncated acquisition, are simply absent from traversal, while structures located in surviving regions continue to be recovered.

Across all of these conditions, the algorithm preserves the property that motivates the present method. It never substitutes interpolated, carved, or inferred content for data it cannot resolve from intact metadata, and every non-recoverable object is either omitted or assigned a failure outcome. This is a deliberate design choice rather than a deficiency to be remedied. Accuracy over a structure whose integrity cannot

be established is, by definition, unverifiable, and attempting recovery in that regime would forfeit the method's determinism and verifiability properties on which its forensic value depends. The contrast with salvage-oriented tools is instructive. An error-skipping mode such as that offered by `btrfs restore` continues past unreadable metadata and copies whatever content remains legible, trading provenance and completeness guarantees for recall. The present method makes the opposite choice, reporting a bounded failure rather than an unverifiable reconstruction. Recovery from genuinely corrupted metadata, as distinct from sound behavior in its presence, remains outside the scope of this work.

H. BTRFS FEATURE COVERAGE

1) Compressed Extents

The algorithm reads the compression type field of each extent data item and decompresses the extent contents using the indicated algorithm before writing the contents to the output file. The same path applies to compressed inline extents.

2) Inline Extents

When an extent item indicates inline storage, file content is read directly from within the leaf node rather than resolved through the chunk tree. Recovery of inline extents depends on the survival of the host leaf, since inline content is not preserved separately in data block groups. This constraint is consistent with the observation by Wani and Bhat that inline-stored small files are vulnerable to overwrite during subsequent node balancing [15].

3) Shared and Relinked Extents

This feature is supported transparently. Files produced by reflink-based copies, deduplication tools, and snapshot creation share physical extents at the logical-address level. Because reconstruction (Algorithm 8) resolves extent data by logical address, reflinked content is reconstructed correctly regardless of how many filesystem objects reference it. A deleted file whose extents remain referenced by another live object remains physically recoverable for as long as the shared extent persists.

4) Snapshots

Each snapshot is treated as an independent subvolume with its own filesystem tree and object identifier space.

5) Sparse Files

Supported. Each extent item carries the file offset at which its data begins (the extent item's key offset), and reconstruction seeks to that offset before writing. Hole regions and prealloc extents are left unwritten during the per-extent loop and materialized as zero-filled bytes by a final truncation to the inode-declared size. This preserves both the sparse layout and the file length recorded in the inode, and the reconstructed file is byte-identical to the original under SHA-256 comparison.

6) Checksummed Extents

Btrfs maintains a checksum tree containing per-extent checksums keyed by logical address. The current implementation validates recovery using SHA-256 comparison against external ground truth, which is appropriate for controlled experiments but unavailable in operational forensic scenarios. Integration of the checksum tree would provide intrinsic verification of recovered extent contents and would convert the overwritten-data-block outcome (currently structurally indistinguishable from complete recovery without external comparison) into a detectable failure condition. Historical checksum tree states preserved through CoW would extend this capability to deleted-extent verification. This integration is identified as future work.

7) Multi-device Btrfs and RAID profiles

Out of scope for true multi-device configurations. The method assumes a single-device raw image as input, with partial coverage of redundancy profiles where the data is fully present on that single device. The chunk parser reads the first stripe of each chunk item, which is sufficient for SINGLE profile (no redundancy) and DUP profile (two copies on the same device, used by default for metadata on single-device installations). For these profiles, recovery proceeds without modification. RAID1, where two copies reside on different devices, is recoverable from a single image only when that image corresponds to the device hosting the first stripe of each affected chunk. RAID0, RAID10, RAID5, and RAID6 profiles require simultaneous access to multiple member devices for logical-to-physical translation and are out of scope. Pool-aware analysis, as described by Hilgert et al. [22], is the appropriate preceding step for multi-device configurations.

For the parity-bearing profiles (RAID5 and RAID6), the surviving stripes constitute erasure-coded redundancy, so reconstruction of missing or overwritten data stripes from available parity is, in principle, an information-theoretic decoding problem rather than a metadata-parsing one. The literature on locally recoverable and regenerating codes for distributed storage [30] provides a principled framework for such reconstruction under partial device availability, combining it with the pool-aware multi-device analysis of Hilgert et al. [22] is a well-defined direction for extending recovery to coded-redundancy profiles.

I. APPLICABILITY CONSTRAINTS

The method assumes a fully valid and correctly structured Btrfs filesystem image. Filesystem images containing structural corruption, incomplete metadata, or inconsistencies introduced by partial writes or hardware failures are outside the scope of this work.

XI. CONCLUSION

This paper presented a systematic, rule-based method for recovering deleted files from Btrfs filesystem images by directly parsing on-disk structures. By taking advantage of

Btrfs's CoW feature and the persistence of historical filesystem trees, the proposed method reconstructs deleted files through logical filesystem analysis rather than heuristic or signature-based carving. Unlike existing approaches, the presented method recovers files, reconstructing their original directory paths and inode metadata alongside available data extents.

The core contribution of this work lies in treating historical filesystem trees as the main forensic artifacts. By identifying and traversing unreferenced filesystem roots preserved across generations, the method distinguishes deleted files from those present in the active filesystem state using object identifiers. This design avoids reliance on orphan-item edge cases or carving techniques. The CoW-aware traversal strategy and node-level caching further ensures efficiency and speed when analyzing multiple historical filesystem states that share common tree nodes.

At the same time, the paper makes clear that recovery is fundamentally constrained by Btrfs's aggressive reuse of metadata blocks, such that once ownership continuity is broken, logical reconstruction for that branch is no longer possible. The suggested algorithm works only on Btrfs images with intact file system structures and does not involve either repairing the file system or recovering from a damaged file system.

Evaluation was conducted on a controlled image and purpose-built scenario images spanning extent layouts, compression, subvolumes and snapshots, classification edge cases, and heavy churn, since no public Btrfs forensic corpus exists at the time of writing. The reported outcomes are therefore indicative of algorithm behavior under the tested structural conditions rather than statistically general recovery rates.

Overall, this work contributes a principled forensic recovery framework grounded in Btrfs's on-disk architecture and transactional semantics. It provides both a functional recovery algorithm and a conceptual foundation for future research on filesystem-aware forensic analysis of CoW filesystems. Potential extensions, such as deep leaf scanning or partial recovery techniques, may improve recall in edge cases but come at the cost of determinism. Within its stated scope, the proposed method establishes that deleted-file recovery in Btrfs can be precise, verifiable, and forensically meaningful when approached through systematic reconstruction rather than carving-based heuristics.

APPENDIX A: AUDIT LOG SCHEMA

The implementation emits a human-readable analysis log during recovery. The log is organized into fixed sections, each documenting one stage of Algorithm 1. This appendix documents the log's structure and shows one representative line per section, drawn from the run on the final image of Table I. The complete log for that run is available as supplementary material.

- 1) **Superblock validation.** Records that the primary superblock was read and parsed, and reports the current filesystem generation G_c extracted from it.


```
[+] Current generation - 244
[+] File opened, superblock read
```
- 2) **Chunk-tree bootstrap.** Reports node and leaf sizes, each system-chunk mapping inserted from the superblock, and the resolved logical and physical addresses of the chunk-tree root. Anomalies encountered during bootstrap (e.g., multi-stripe chunks on a single-device image) are recorded as warnings.


```
[*] Bootstrapping chunk tree
Node size: 16384; Leaf size: 16384
[-] warning: 2 stripes detected,
    processing 1
[+] Chunk inserted: start=22020096,
    size=8388608
chunk tree root: logical=22200320,
    physical=0x152c000
```
- 3) **Root-tree and filesystem-tree location.** Records the logical and physical addresses of the root tree and the default filesystem-tree root, both resolved through the bootstrapped chunk tree.


```
root tree root: logical=133316608,
    physical=0x8724000
fs tree root:   logical=30638080,
    physical=0x2538000
```
- 4) **Subvolume enumeration.** Lists each subvolume discovered in the root tree by name and object identifier. Subvolumes outside the recovery scope are marked accordingly.


```
[*] Subvolume home (objectid=262)
[*] Subvolume var   (objectid=257)
    [skipped]
```
- 5) **Historical-root discovery.** Records the generation-indexed map produced by Algorithm 3, associating each discovered historical root tree with its generation number and physical offset.


```
OLD ROOTS (generation :
    physical_address)
{106: 0x35c8000, 204: 0x3690000,
 ..., 244: 0x8724000}
```
- 6) **Deleted-file candidate identification.** For the subvolume under analysis, lists each object identifier classified as deleted by the predicate of Section IV.B, together with the recovered file name. Entries skipped due to ownership ambiguity (see Section VII.E) are reported separately.


```
[+] Deleted items in subvolume home:
    file1.txt, video2.mp4,
    filex.txt, binary.bin,
    image5.jpg, video1.mkv,
    audio2.mp3, audio1.wav,
    image3.bmp, .xsession-errors,
```

```
.xsession-errors.old
[---] entries skipped:
      owner ID ambiguity
```

- 7) **Per-file reconstruction record.** For each deleted file, records the reconstructed path, logical size, inode timestamps, and extent layout (inline, single-extent, or multi-extent). Files whose extents cannot be fully resolved through the current chunk tree are annotated with the failure cause from the taxonomy.

```
Path: home/user/.../file1.txt
Size: 1000 bytes
Layout: inline
atime/mtime/ctime/otime:
      2026-02-10 ...
Path: home/user/.../video1.mkv
Size: 1073741824 bytes
Layout: multi-extent
Outcome: partial
Cause: missing chunk mapping
```

- 8) **Run summary.** Records total wall-clock recovery time and a count of files emitted per outcome category.

```
[+] Recovery time: 4.4809 s
```

The per-file records emitted in section 7 of the log correspond directly to the entries in Table II, and the outcome and failure-cause fields use the taxonomy. An auditor reproducing the analysis on the same image can verify every decision made by the algorithm from the log alone, without re-executing the tool.

FUNDING

The publication of this article was supported by Manipal Academy of Higher Education, Manipal -576104, Karnataka -India.

ACKNOWLEDGMENT

The authors acknowledge Arpit Arora, a member of the Student Project “CRYPTONITE”, for his substantial contributions to manuscript formatting, and helpful review, and the entire team of “CRYPTONITE” for their valuable insights during the course of the study.

REFERENCES

- [1] O. Rodeh, J. Bacik, and C. Mason, “BTRFS: The Linux B-tree filesystem,” *ACM Trans. Storage*, vol. 9, no. 3, pp. 1–32, Oct. 2013, doi: 10.1145/2501620.2501623.
- [2] Linux Kernel Documentation, “BTRFS — The Linux Kernel documentation,” Accessed: Jan. 1, 2024. [Online]. Available: <https://docs.kernel.org/filesystems/btrfs.html>.
- [3] F. Buchholz and E. Spafford, “On the role of file system metadata in digital forensics,” *Digit. Investig.*, vol. 1, no. 4, pp. 298–309, Dec. 2004, doi: 10.1016/j.diin.2004.10.002.
- [4] C. Hargreaves and J. Patterson, “An automated timeline reconstruction approach for digital forensic investigations,” *Digit. Investig.*, vol. 9, pp. S69–S79, Aug. 2012, doi: 10.1016/j.diin.2012.05.006.
- [5] A. Pal and N. Memon, “The evolution of file carving,” *IEEE Signal Process. Mag.*, vol. 26, no. 2, pp. 59–71, Feb. 2009, doi: 10.1109/MSP.2008.931081.
- [6] Btrfs Wiki, “Data structures,” Accessed: Jan. 1, 2024. [Online]. Available: https://archive.kernel.org/oldwiki/btrfs.wiki.kernel.org/index.php/Data_Structures.html.
- [7] —, “On-disk format,” Accessed: Jan. 1, 2024. [Online]. Available: https://archive.kernel.org/oldwiki/btrfs.wiki.kernel.org/index.php/On-disk_Format.html.
- [8] N. L. Beebe, S. D. Stacy, and D. Stuckey, “Digital forensic implications of ZFS,” *Digit. Investig.*, vol. 6, pp. S99–S107, Sep. 2009, doi: 10.1016/j.diin.2009.06.006.
- [9] O. Rodeh, “B-trees, shadowing, and clones,” *ACM Trans. Storage*, vol. 3, no. 4, pp. 1–27, Feb. 2008, doi: 10.1145/1326542.1326544.
- [10] A. Juch, “Btrfs filesystem forensics,” Diploma thesis, Dept. Comput. Eng., Technische Universität Wien, Vienna, Austria, 2014, doi: 10.34726/hss.2014.24504.
- [11] J. Chen, X. Liao, W. Wang, Z. Qian, Z. Qin, and Y. Wang, “SNIS: A signal noise separation-based network for post-processed image forgery detection,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 2, pp. 935–951, Feb. 2023, doi: 10.1109/TCSVT.2022.3204753.
- [12] X. Liao, Y. Wang, T. Wang, J. Hu, and X. Wu, “FAMM: Facial muscle motions for detecting compressed deepfake videos over social networks,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 12, pp. 7236–7251, Dec. 2023, doi: 10.1109/TCSVT.2023.3278310.
- [13] L. Fu, X. Liao, J. Guo, L. Dong, and Z. Qin, “WaveRecovery: Screen-shooting watermarking based on wavelet and recovery,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, pp. 3603–3618, Apr. 2025, doi: 10.1109/TCSVT.2024.3510355.
- [14] Y. Li, X. Liao, and X. Wu, “Screen-shooting resistant watermarking with grayscale deviation simulation,” *IEEE Trans. Multimedia*, vol. 26, pp. 10908–10923, Jan. 2024, doi: 10.1109/TMM.2024.3415415.
- [15] W. A. Bhat and M. A. Wani, “Forensic analysis of B-tree file system (Btrfs),” *Digit. Investig.*, vol. 27, pp. 57–70, Dec. 2018, doi: 10.1016/j.diin.2018.09.001.
- [16] M. A. Wani, W. A. Bhat, and A. Dehghantanha, “An analysis of anti-forensic capabilities of B-tree file system (Btrfs),” *Aust. J. Forensic Sci.*, vol. 52, no. 4, pp. 371–386, Jul. 2020, doi: 10.1080/00450618.2018.1533038.
- [17] M. A. Wani and W. A. Bhat, “Dataset for forensic analysis of B-tree file system,” *Data Brief*, vol. 18, pp. 2013–2018, Jun. 2018, doi: 10.1016/j.dib.2018.04.100.
- [18] K. D. Fairbanks, “An analysis of Ext4 for digital forensics,” *Digit. Investig.*, vol. 9, pp. S118–S130, Aug. 2012, doi: 10.1016/j.diin.2012.05.003.
- [19] A. Dewald and S. Seufert, “AFEIC: Advanced forensic Ext4 inode carving,” *Digit. Investig.*, vol. 20, pp. S83–S91, Mar. 2017, doi: 10.1016/j.diin.2017.01.003.
- [20] B. Carrier, *File System Forensic Analysis*. Upper Saddle River, NJ, USA: Addison-Wesley, 2005.
- [21] H. Kim, S. Kim, Y. Shin, W. Jo, S. Lee, and T. Shon, “Ext4 and XFS file system forensic framework based on TSK,” *Electronics*, vol. 10, no. 18, p. 2310, Sep. 2021, doi: 10.3390/electronics10182310.
- [22] J.-N. Hilgert, M. Lambertz, and S. Yang, “Forensic analysis of multiple device BTRFS configurations using The Sleuth Kit,” *Digit. Investig.*, vol. 26, pp. S21–S29, Aug. 2018, doi: 10.1016/j.diin.2018.04.020.
- [23] J.-N. Hilgert, M. Lambertz, and D. Plohmann, “Extending The Sleuth Kit and its underlying model for pooled storage file system forensic analysis,” *Digit. Investig.*, vol. 22, pp. S76–S85, Aug. 2017, doi: 10.1016/j.diin.2017.06.014.
- [24] btrfs-restore (8) – Linux Manual Page, man7.org, 2025, accessed: 2026-06-11. [Online]. Available: <https://man7.org/linux/man-pages/man8/btrfs-restore.8.html>
- [25] S. L. Garfinkel, “Carving contiguous and fragmented files with fast object validation,” *Digit. Investig.*, vol. 4, no. S1, pp. 2–12, Sep. 2007, doi: 10.1016/j.diin.2007.06.017.
- [26] H. J. Hadi, Y. Cao, and K. U. Nisa, “SSD forensic: Evidence generation and forensic research on solid state drives using TRIM analysis,” *arXiv preprint arXiv:2307.10192*. [Online]. Available: <https://arxiv.org/abs/2307.10192>, 2023.
- [27] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.
- [28] Linux Kernel Source, “btrfs_get_free_objectid(),” Accessed: Jun. 3, 2026. [Online]. Available: <https://github.com/torvalds/linux/blob/master/fs/btrfs/disk-io.c>.
- [29] National Institute of Standards and Technology, “Secure hash standard (SHS),” NIST, Gaithersburg, MD, USA, Tech. Rep. FIPS PUB 180-4, 2015.

- [30] S. Kruglik and A. Frolov, "An information-theoretic approach for reliable distributed storage systems," *J. Commun. Technol. Electron.*, vol. 65, no. 12, pp. 1505–1516, Dec. 2020, doi: 10.1134/S1064226920120116.



KRISH PANDEY is pursuing the B.Tech. degree in cyber-physical systems from Manipal Institute of Technology, Manipal, India, expected in 2027. He was a Security Research Intern at the National Technical Research Organization (NTRO). He is currently Head of Attack Defense and Hardware Diagnostics (ADHD) at the CTF team Cryptonite, Manipal Institute of Technology, Manipal, India. His research interests include hardware security, file system internals, pentesting, and reverse engineering. He is an active Capture the Flag player. He led the Cryptonite team to first place in the India region at the CSAW Embedded Security Challenge (ESC) 2025.



MISHA JAIN is a B.Tech. student specializing in Artificial Intelligence and Machine Learning at Manipal Institute of Technology, Manipal, India. She is an incoming Software Engineering Intern at Google (Summer 2026) and has previously worked as a Forensic Research Intern at the National Technical Research Organisation (NTRO). She has worked on machine learning systems, including a textual entailment-based medical diagnosis assistant integrated with Rasa and AllenNLP. She is actively involved in cybersecurity research and Capture the Flag (CTF) competitions as part of the Web Exploitation Board of Cryptonite, the cybersecurity team at Manipal Institute of Technology. She was a winner in the forensic category at the Smart India Hackathon (SIH) 2024. Her research interests include natural language processing, cybersecurity, machine learning systems, and intelligent developer tools.



NISHA P. SHETTY Dr. Nisha P. Shetty is a dedicated academician currently serving as an Associate Professor in the School of Computer Engineering at Manipal Institute of Technology (MIT), a part of Manipal Academy of Higher Education (MAHE). With a solid educational background, Dr. Shetty earned her Ph.D. in Data Privacy and Security from MIT, MAHE in 2023, an M.Tech in Computer Science and Engineering from Visvesvaraya Technological University (VTU) in 2015, and a B.E in Computer Science and Engineering from VTU in 2013. Additionally, Dr. Shetty serves as a faculty advisor for Project Cryptonite, one of the top-performing student projects at MIT, which has been winning accolades nationally and internationally. Her research interests include data privacy and security, with a focus on privacy-preserving frameworks in social networks, deep learning for medical diagnostics, and enhanced security measures for online data.

• • •